

Neural Networks

Problem: Linear Classifiers are not very powerful

Problem: Linear Classifiers are not very powerful

Visual Viewpoint



Linear classifiers learn
one template per class

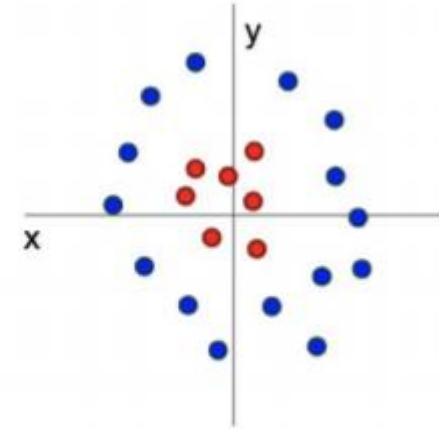
Problem: Linear Classifiers are not very powerful

Visual Viewpoint



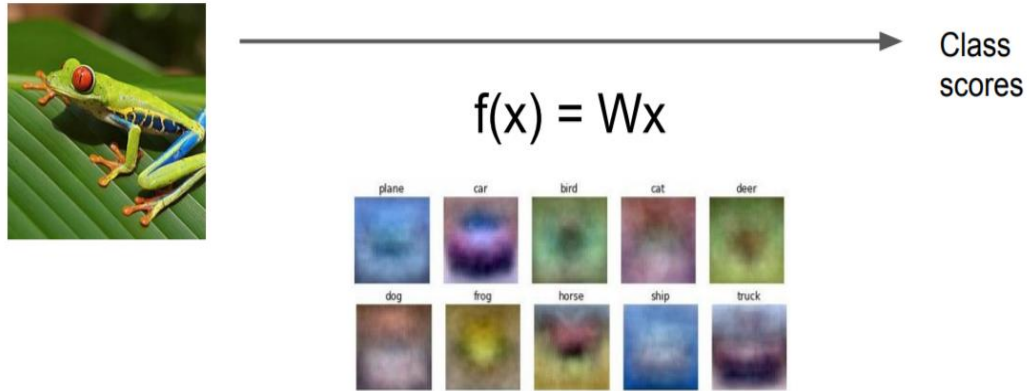
Linear classifiers learn
one template per class

Geometric Viewpoint



Linear classifiers
can only draw linear
decision boundaries

Pixel Features vs Image Features



Pixel Features vs Image Features

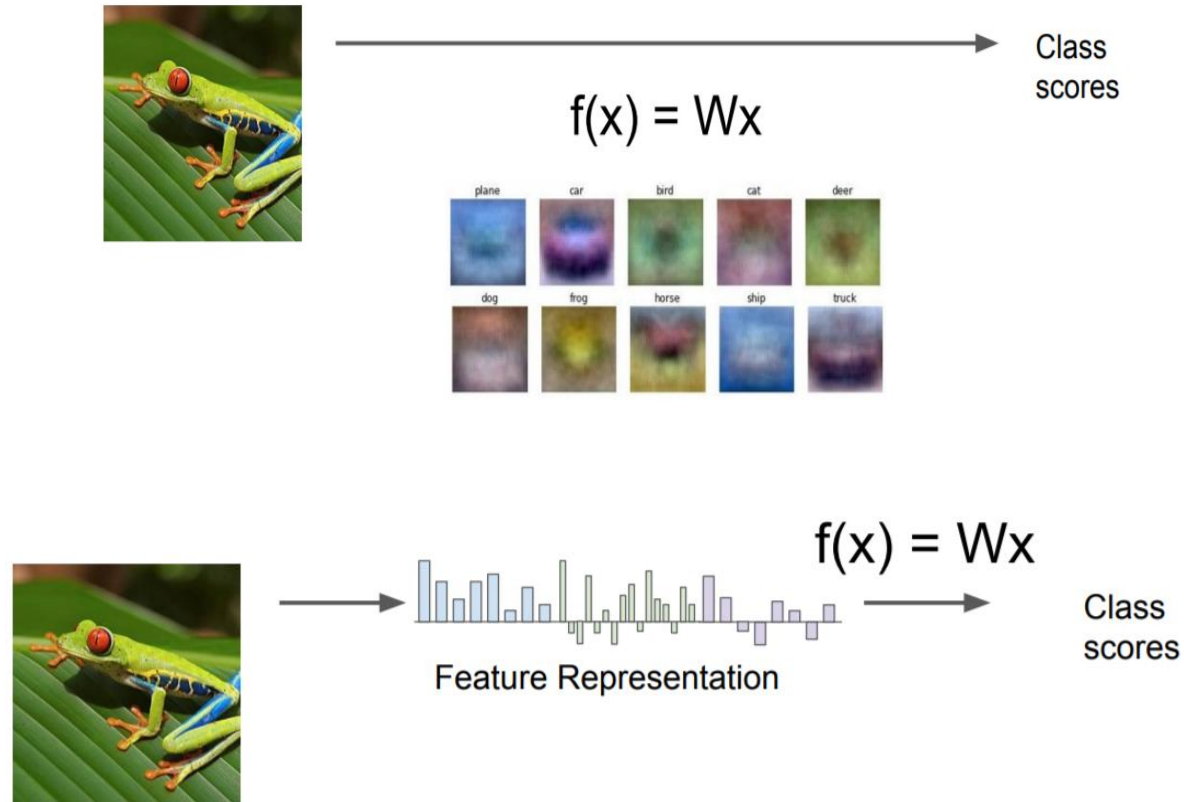
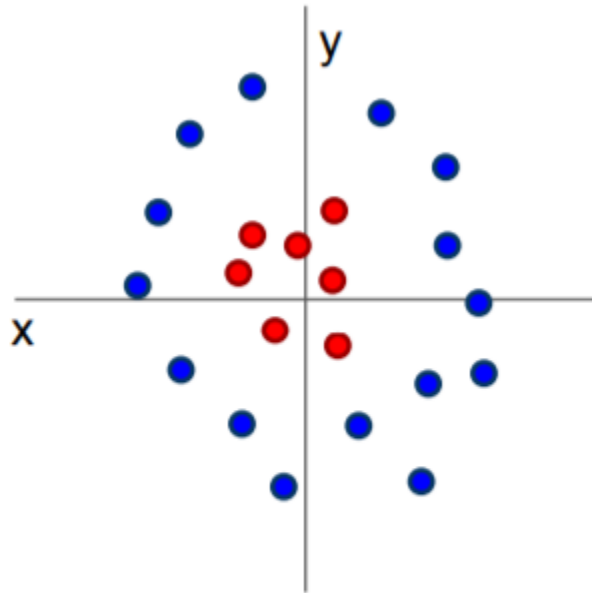
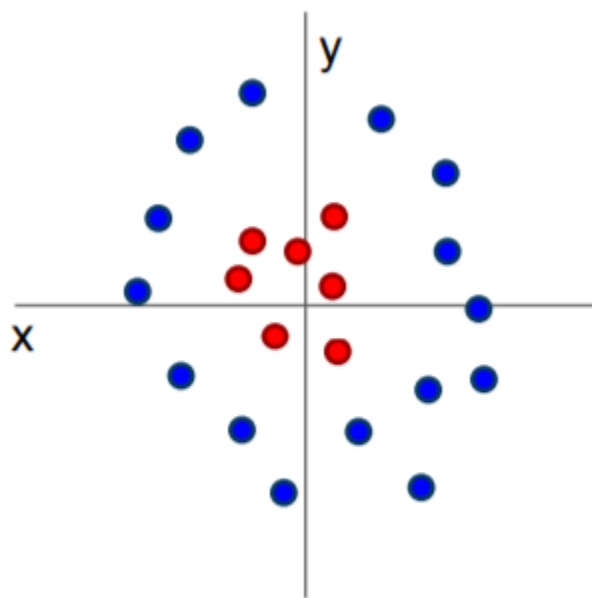


Image Features: Motivation



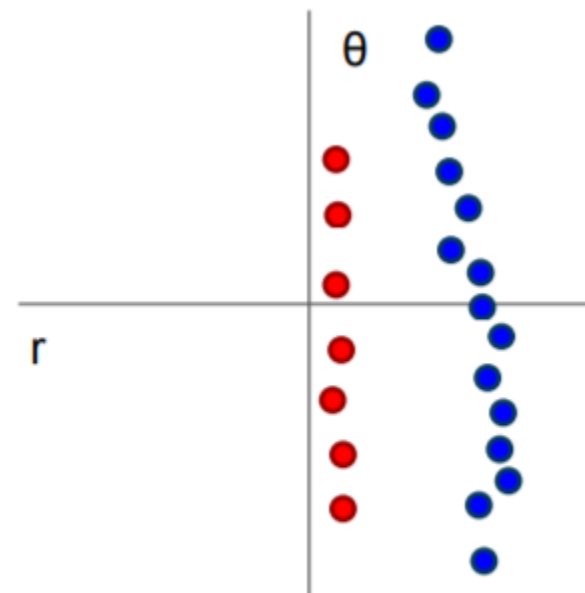
Cannot separate red
and blue points with
linear classifier

Image Features: Motivation



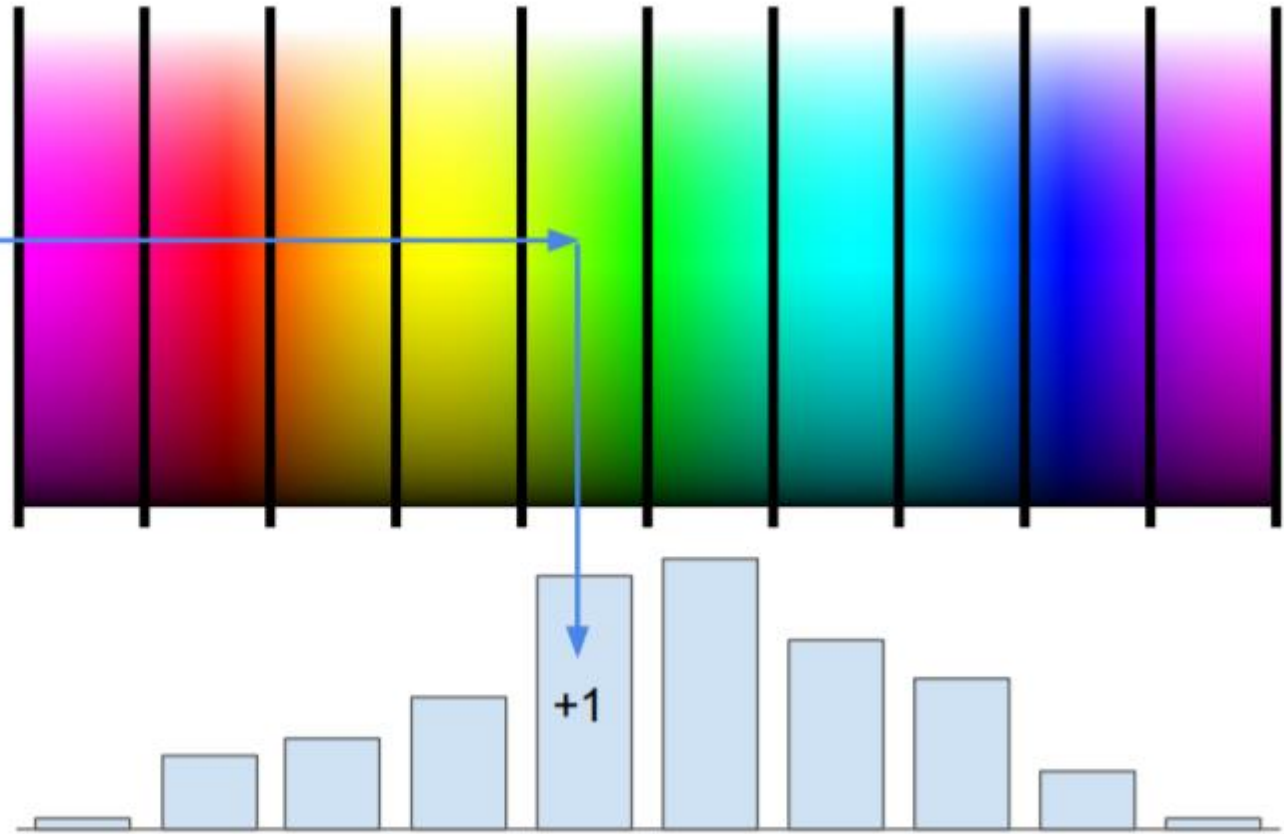
Cannot separate red and blue points with linear classifier

$$f(x, y) = (r(x, y), \theta(x, y))$$



After applying feature transform, points can be separated by linear classifier

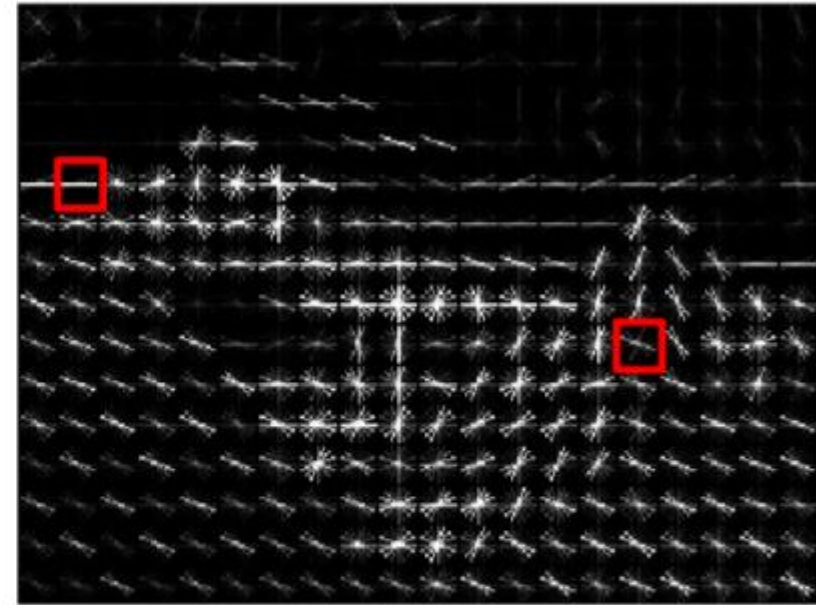
Color Histogram



Histogram of Oriented Gradients (HoG)



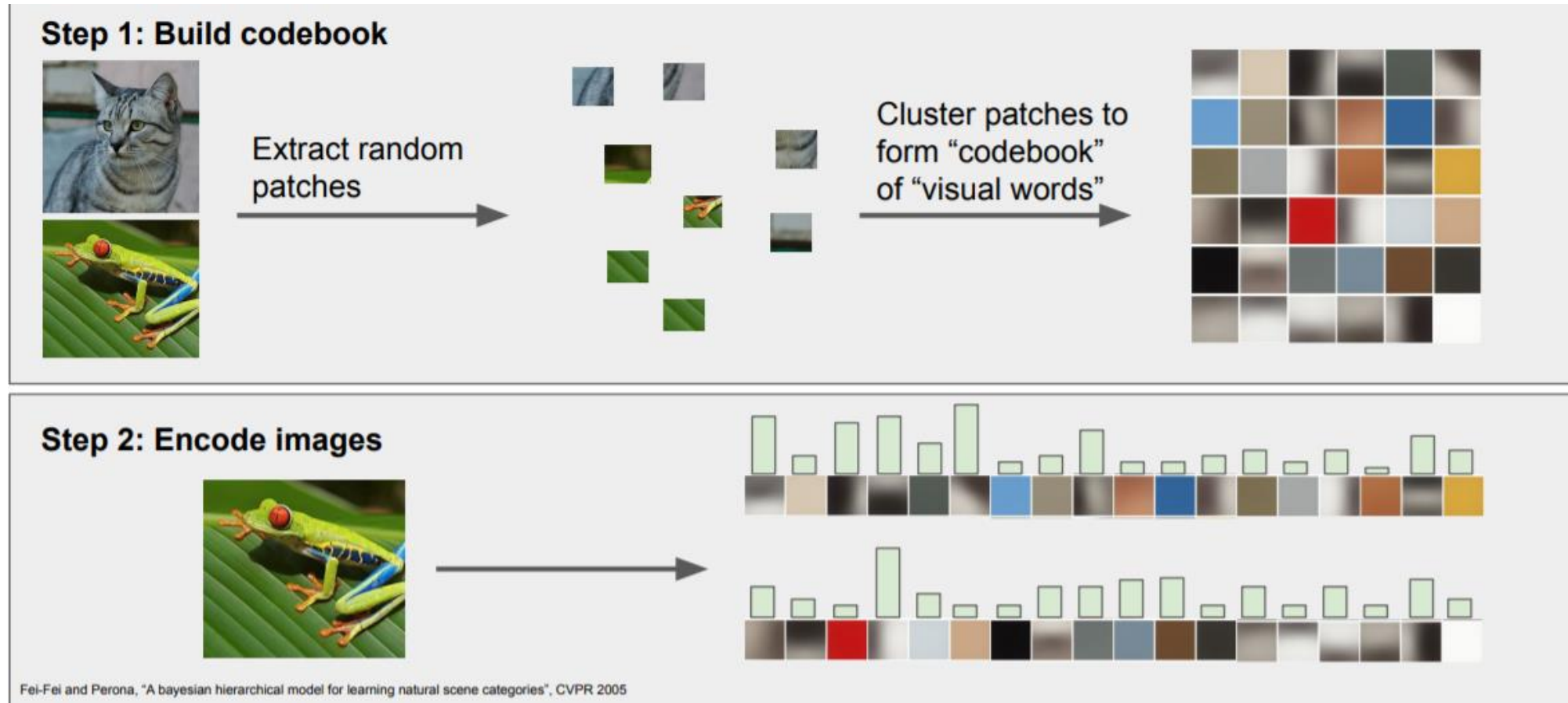
Divide image into 8x8 pixel regions
Within each region quantize edge
direction into 9 bins



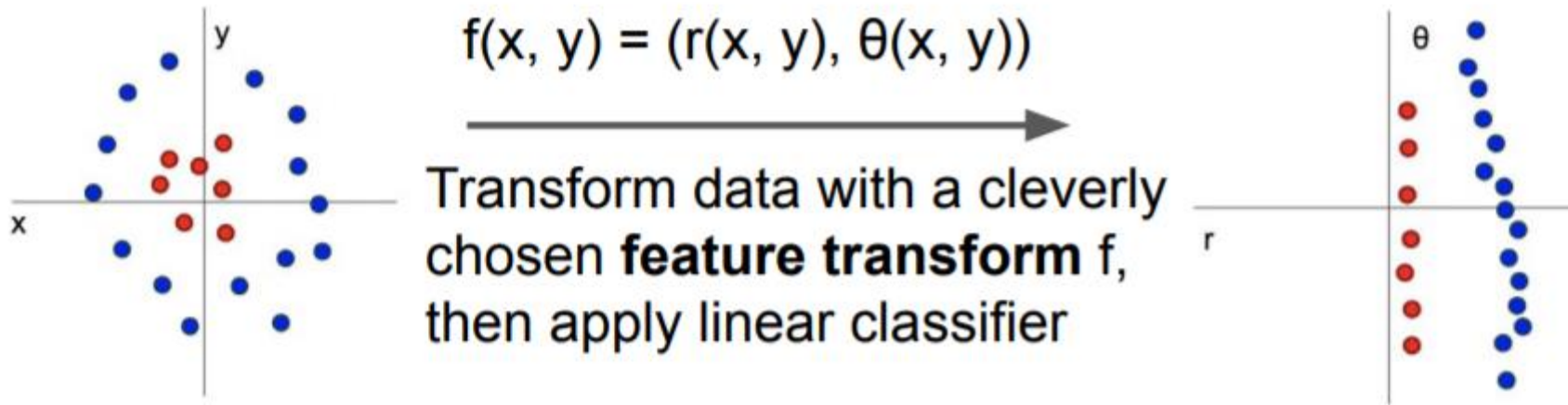
Example: 320x240 image gets divided
into 40x30 bins; in each bin there are
9 numbers so feature vector has
 $30 \times 40 \times 9 = 10,800$ numbers

Lowe, "Object recognition from local scale-invariant features", ICCV 1999
Dalal and Triggs, "Histograms of oriented gradients for human detection," CVPR 2005

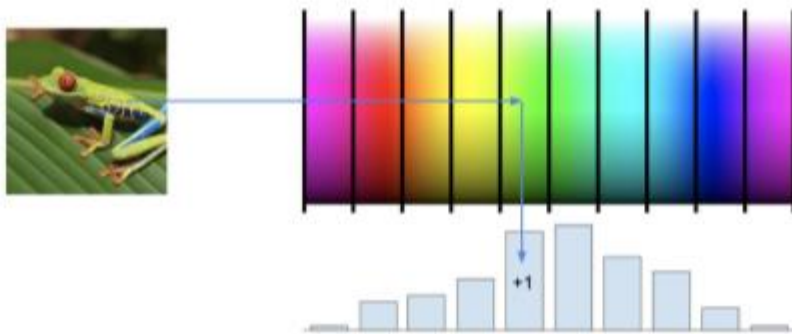
Bag of Words



One Solution: Feature Transformation



Color Histogram



Histogram of Oriented Gradients (HoG)

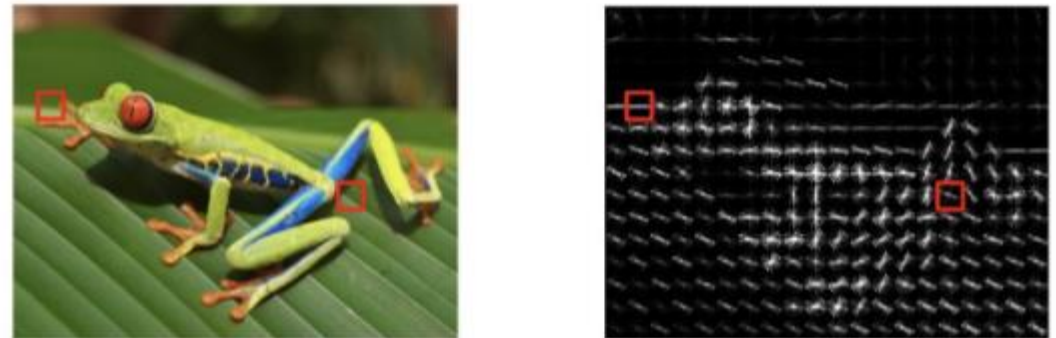
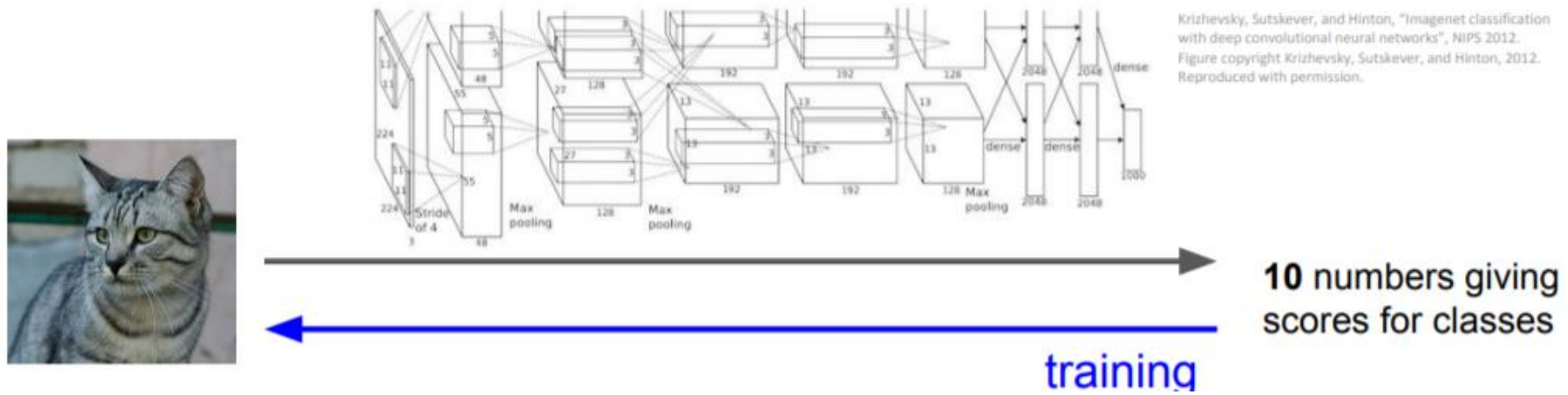
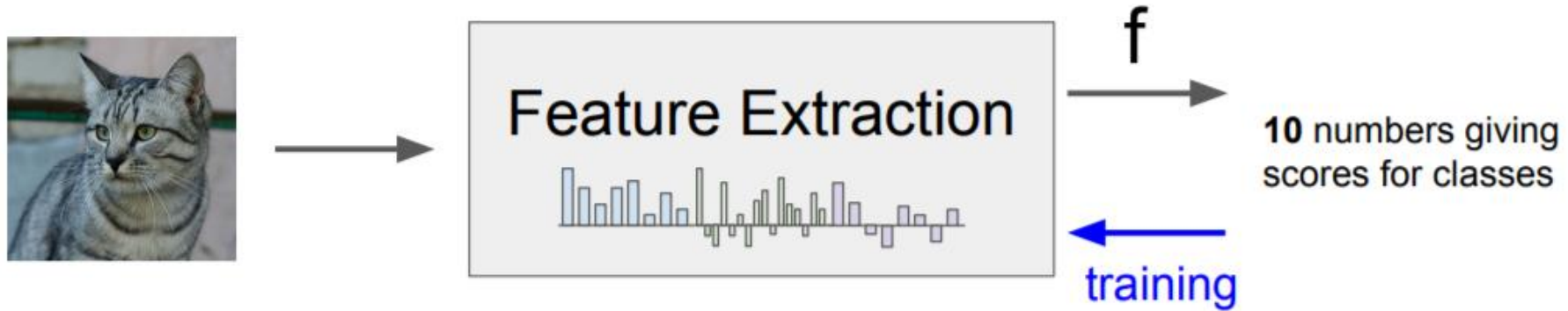


Image features vs ConvNets



Linear Score Functions?

- Can we make linear regression better?

Linear Score Functions?

- Can we make linear regression better?
 - Multiply with another weight matrix $\mathbf{W}_2$

Linear Score Functions?

- Can we make linear regression better?
 - Multiply with another weight matrix $\mathbf{W}_2$

$$\hat{\mathbf{f}} = \mathbf{W}_2 \cdot \mathbf{f}$$

$$\hat{\mathbf{f}} = \mathbf{W}_2 \cdot \mathbf{W} \cdot \mathbf{x}$$

Linear Score Functions?

- Can we make linear regression better?
 - Multiply with another weight matrix $\mathbf{W}_2$

$$\hat{\mathbf{f}} = \mathbf{W}_2 \cdot \mathbf{f}$$

$$\hat{\mathbf{f}} = \mathbf{W}_2 \cdot \mathbf{W} \cdot \mathbf{x}$$

- Operation is still linear.

$$\hat{\mathbf{W}} = \mathbf{W}_2 \cdot \mathbf{W}$$

$$\hat{\mathbf{f}} = \hat{\mathbf{W}} \mathbf{x}$$

Linear Score Functions?

- Can we make linear regression better?
 - Multiply with another weight matrix $\mathbf{W}_2$

$$\hat{\mathbf{f}} = \mathbf{W}_2 \cdot \mathbf{f}$$

$$\hat{\mathbf{f}} = \mathbf{W}_2 \cdot \mathbf{W} \cdot \mathbf{x}$$

- Operation is still linear.

$$\hat{\mathbf{W}} = \mathbf{W}_2 \cdot \mathbf{W}$$

$$\hat{\mathbf{f}} = \hat{\mathbf{W}} \mathbf{x}$$

- Solution $\rightarrow$ add non-linearity!!

Neural Network

- Linear score function $f = Wx$

Neural Network

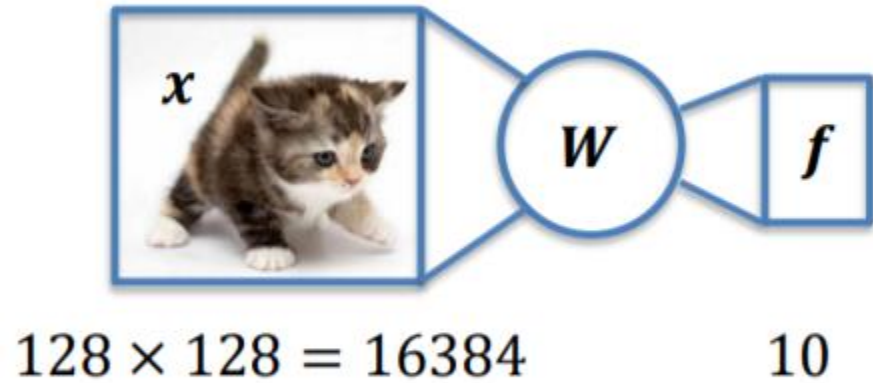
- Linear score function $\mathbf{f} = \mathbf{W}\mathbf{x}$
- Neural network is a nesting of 'functions'
 - 2-layers: $\mathbf{f} = \mathbf{W}_2 \max(\mathbf{0}, \mathbf{W}_1\mathbf{x})$

Neural Network

- Linear score function $\mathbf{f} = \mathbf{W}\mathbf{x}$
- Neural network is a nesting of 'functions'
 - 2-layers: $\mathbf{f} = \mathbf{W}_2 \max(\mathbf{0}, \mathbf{W}_1 \mathbf{x})$
 - 3-layers: $\mathbf{f} = \mathbf{W}_3 \max(\mathbf{0}, \mathbf{W}_2 \max(\mathbf{0}, \mathbf{W}_1 \mathbf{x}))$
 - 4-layers: $\mathbf{f} = \mathbf{W}_4 \tanh(\mathbf{W}_3, \max(\mathbf{0}, \mathbf{W}_2 \max(\mathbf{0}, \mathbf{W}_1 \mathbf{x})))$
 - 5-layers: $\mathbf{f} = \mathbf{W}_5 \sigma(\mathbf{W}_4 \tanh(\mathbf{W}_3, \max(\mathbf{0}, \mathbf{W}_2 \max(\mathbf{0}, \mathbf{W}_1 \mathbf{x}))))$
 - ... up to hundreds of layers

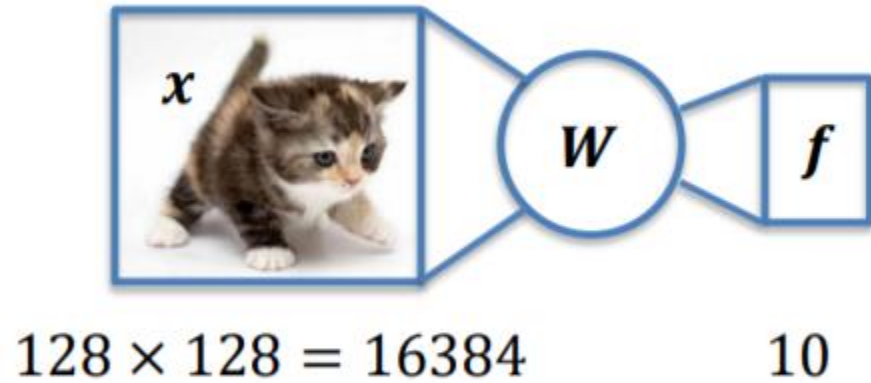
Neural Network

1-layer network: $f = Wx$

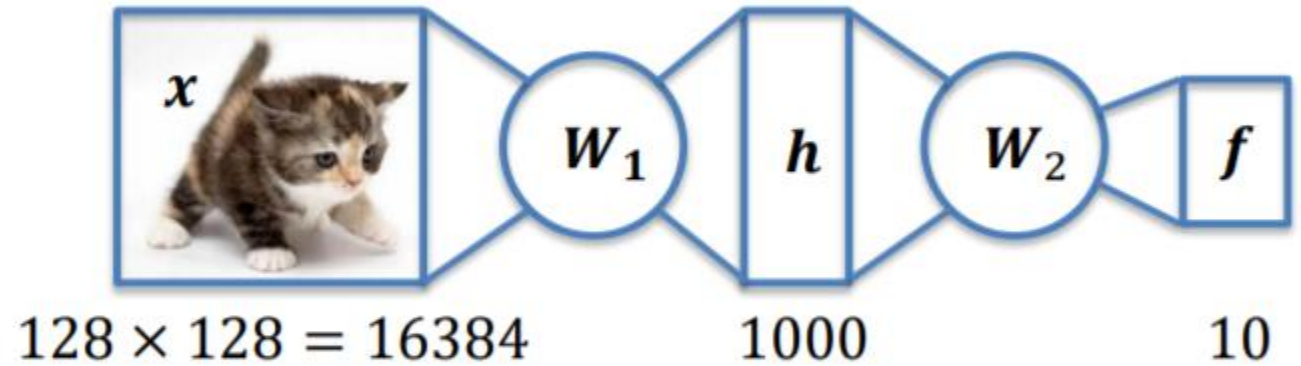


Neural Network

1-layer network: $f = Wx$

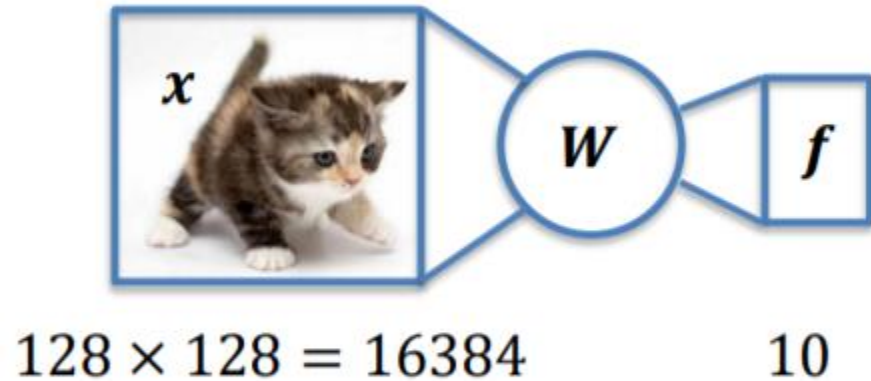


2-layer network: $f = W_2 \max(0, W_1 x)$

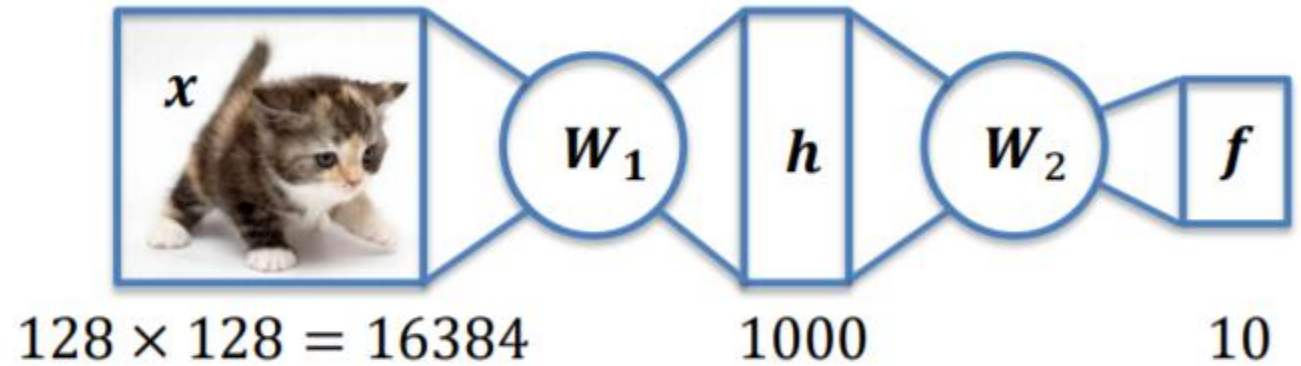


Neural Network

1-layer network: $f = Wx$

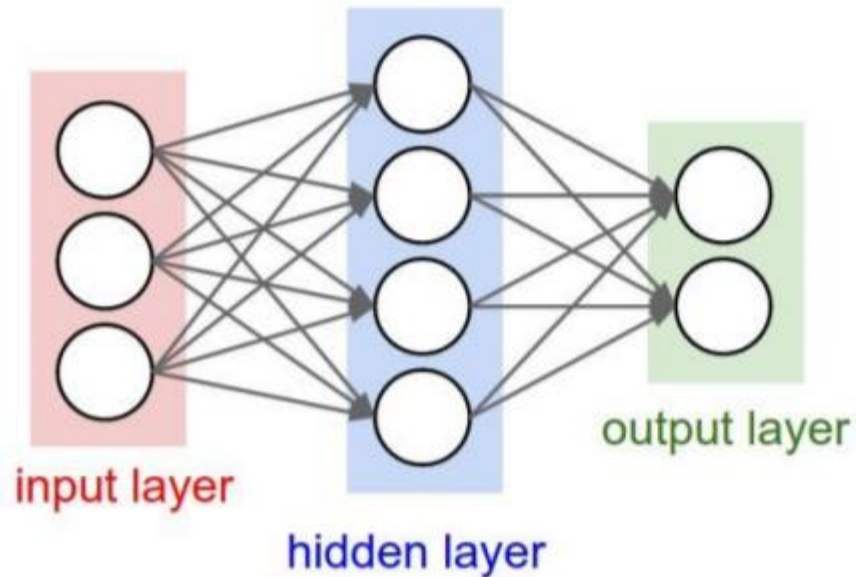


2-layer network: $f = W_2 \max(0, W_1 x)$

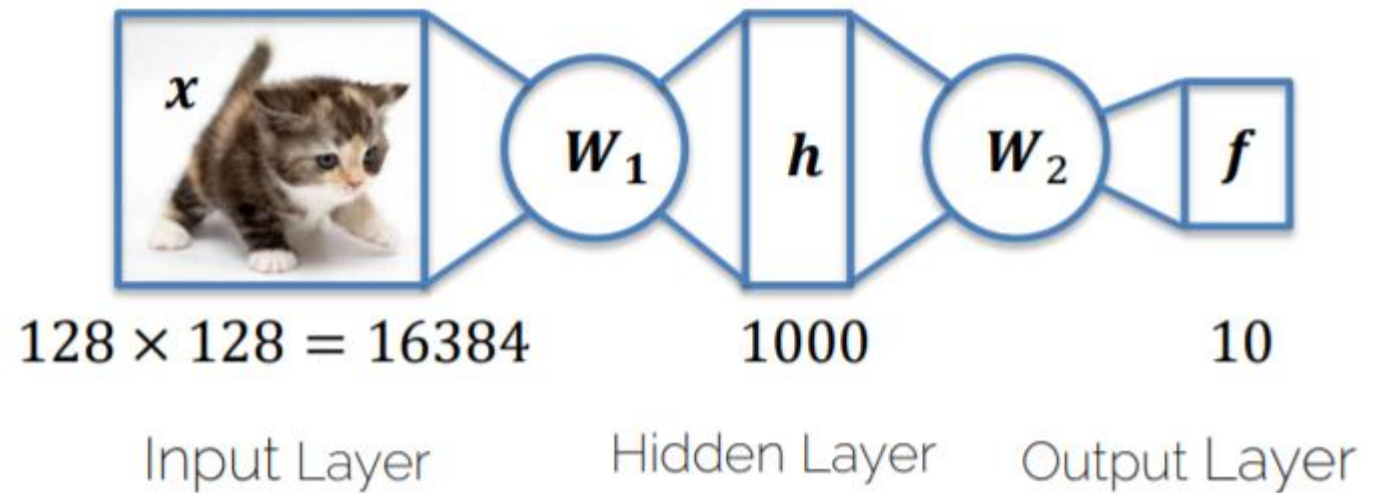


Why is this structure useful?

Neural Network

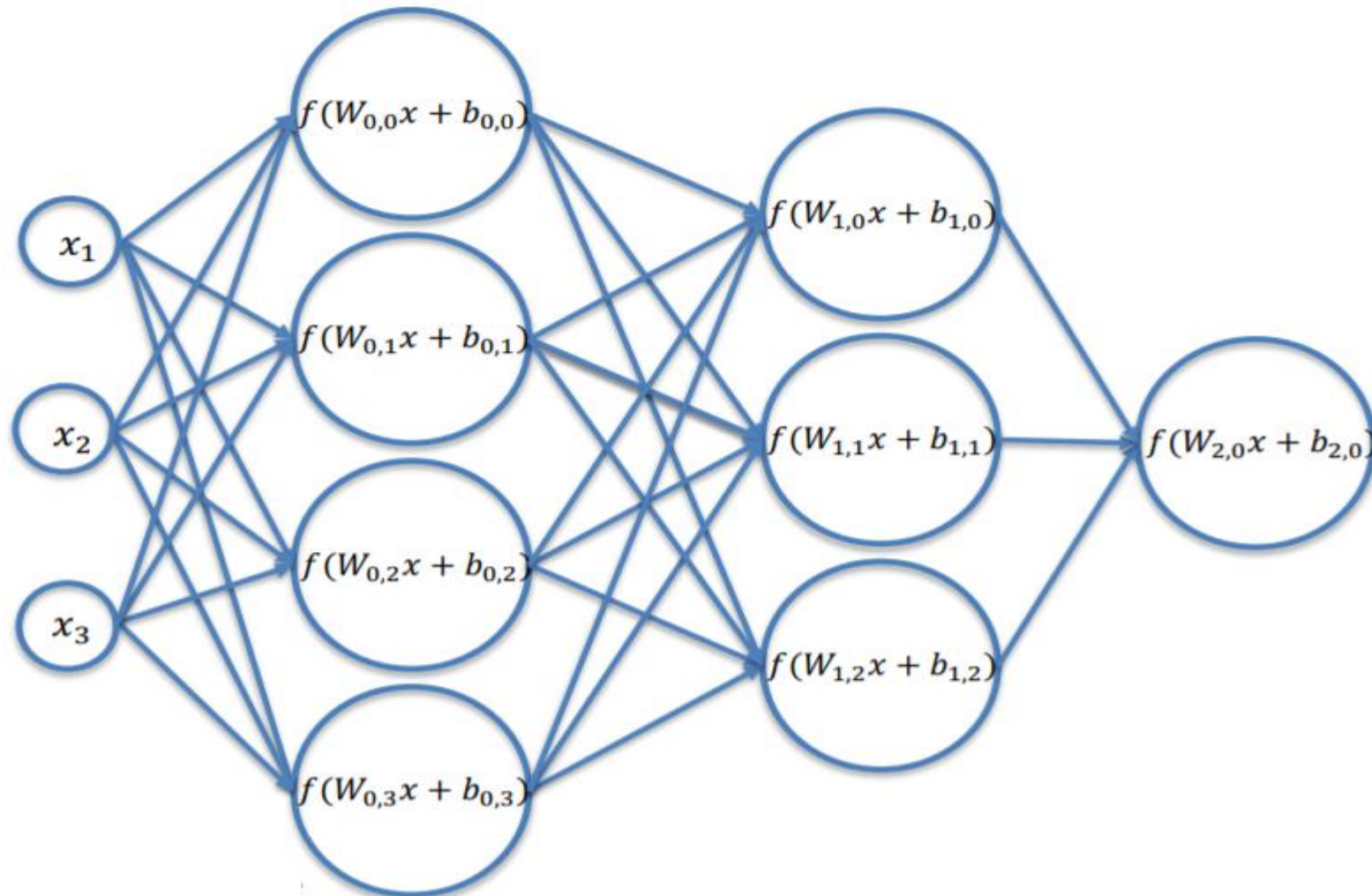


2-layer network: $f = W_2 \max(0, W_1 x)$

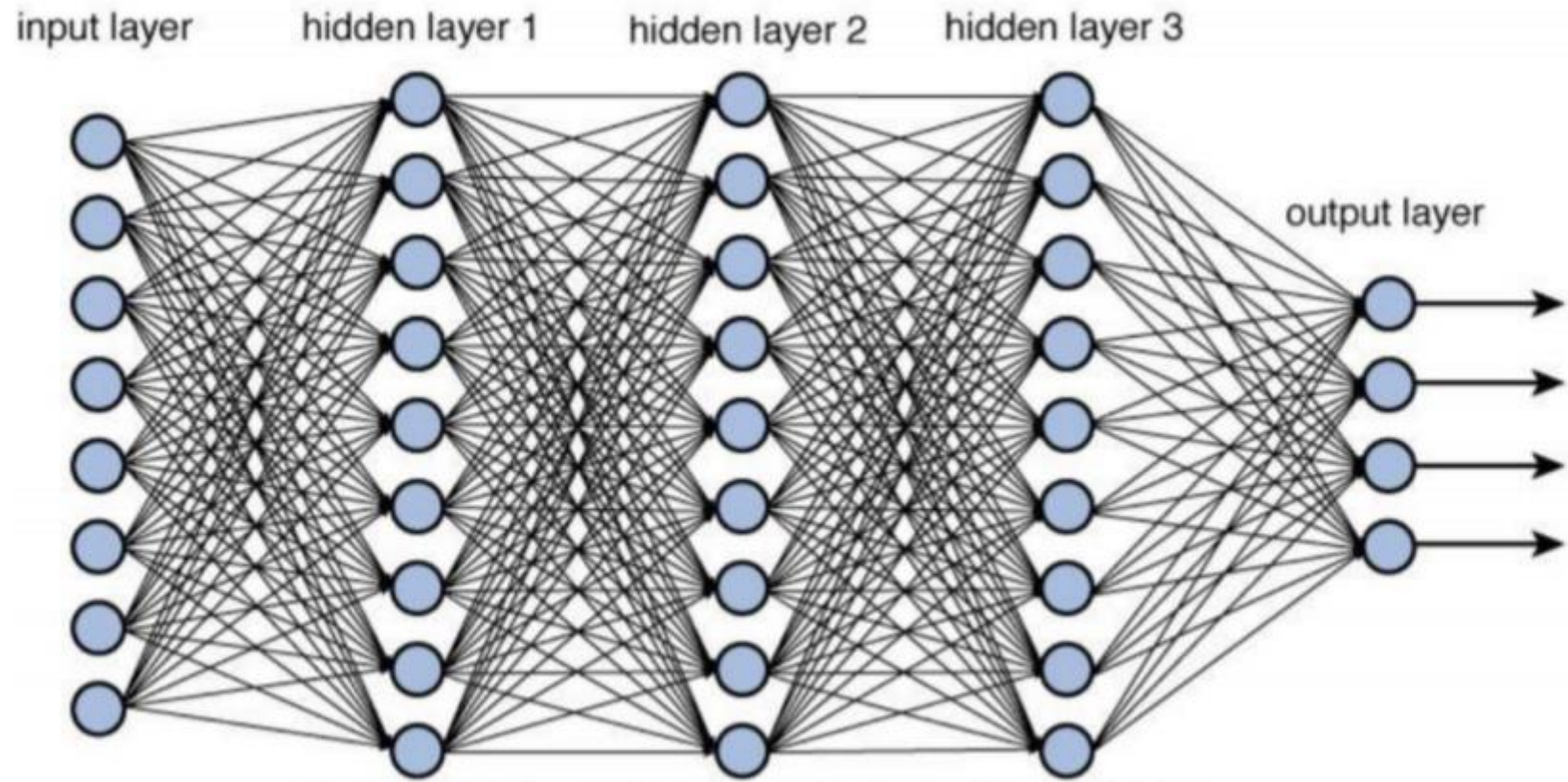


Learn 1000 templates instead of 10. Share templates between classes

Net of Artificial Neurons



Neural Network

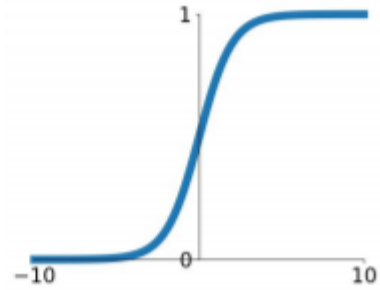


Source: <https://towardsdatascience.com/training-deep-neural-networks-gfdb1964b964>

Activation Functions

Sigmoid

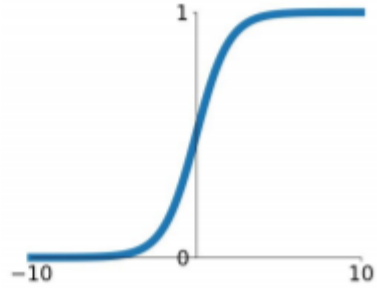
$$\sigma(x) = \frac{1}{1+e^{-x}}$$



Activation Functions

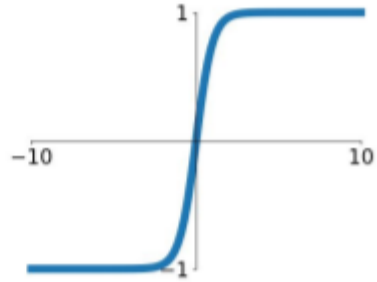
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



tanh

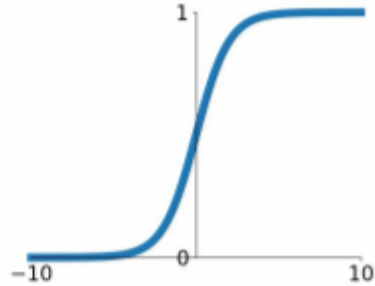
$$\tanh(x)$$



Activation Functions

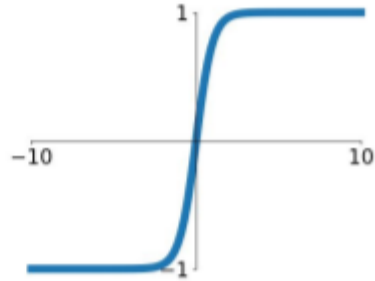
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



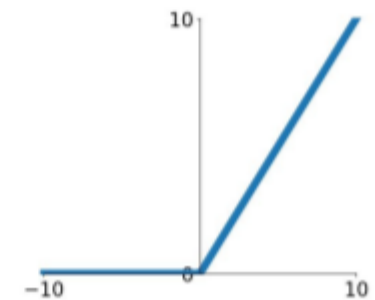
tanh

$$\tanh(x)$$



ReLU

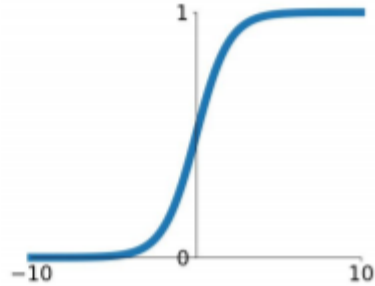
$$\max(0, x)$$



Activation Functions

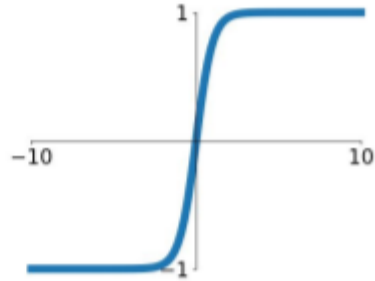
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



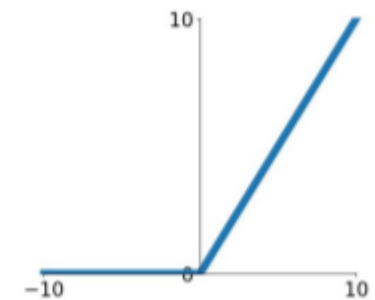
tanh

$$\tanh(x)$$



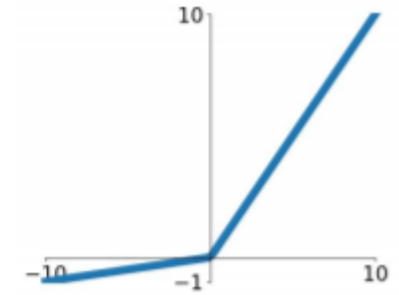
ReLU

$$\max(0, x)$$



Leaky ReLU

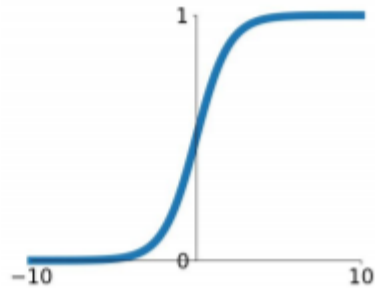
$$\max(0.1x, x)$$



Activation Functions

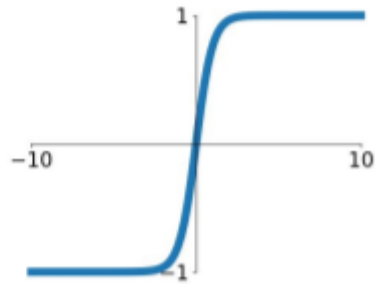
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



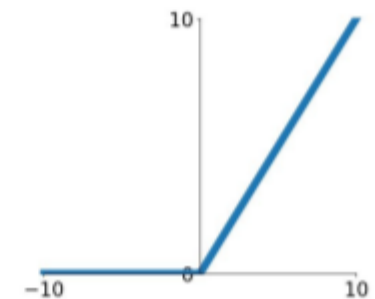
tanh

$$\tanh(x)$$



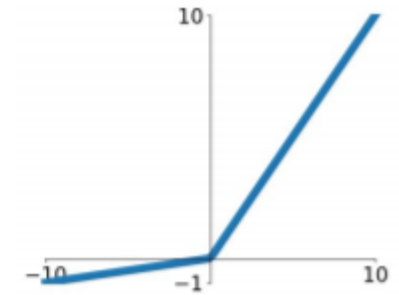
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$



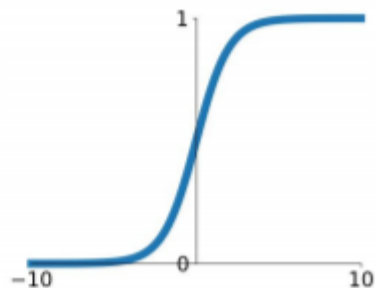
Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

Activation Functions

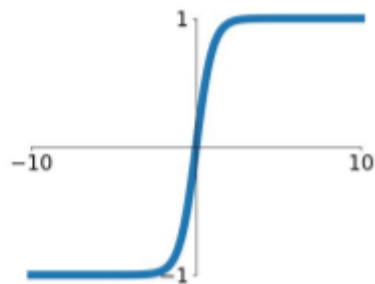
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



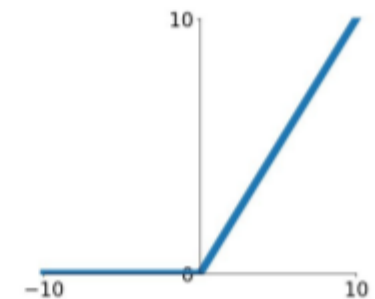
tanh

$$\tanh(x)$$



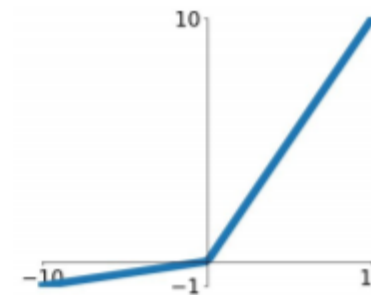
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

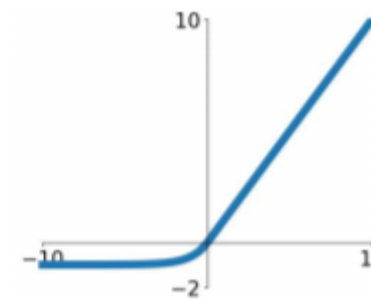


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

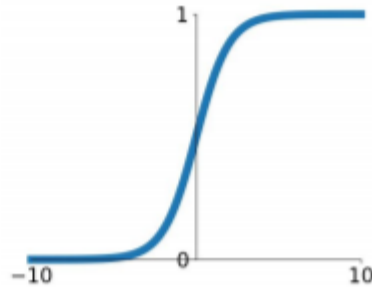
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Activation Functions

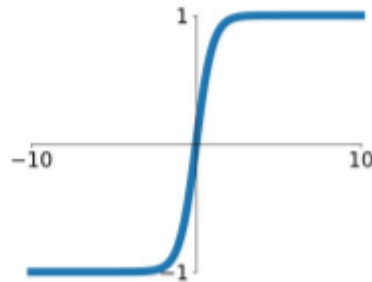
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



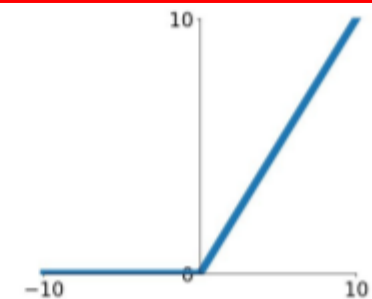
tanh

$$\tanh(x)$$



ReLU

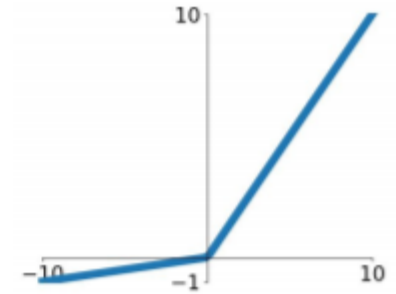
$$\max(0, x)$$



ReLU (rectified linear unit) is a good default choice for most problems

Leaky ReLU

$$\max(0.1x, x)$$

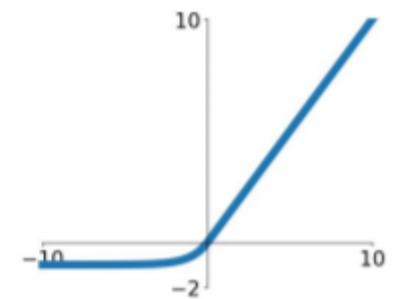


Maxout

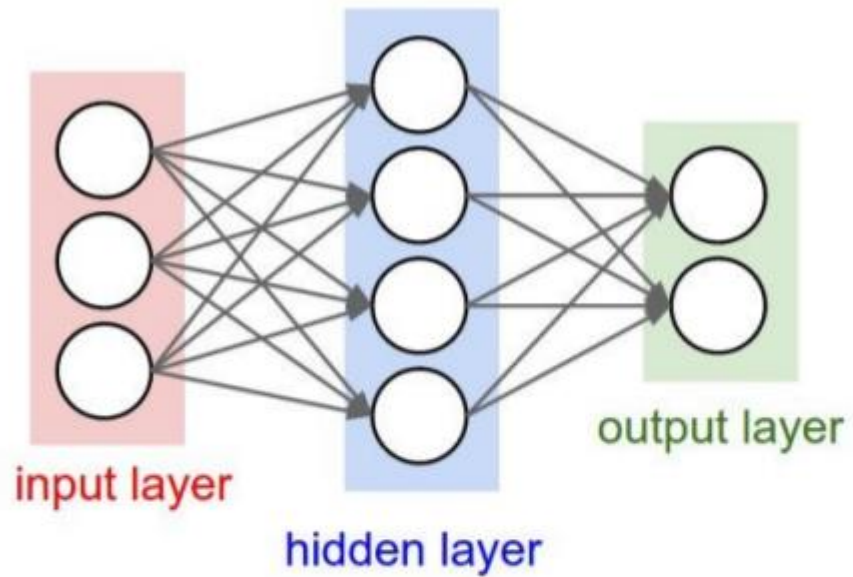
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

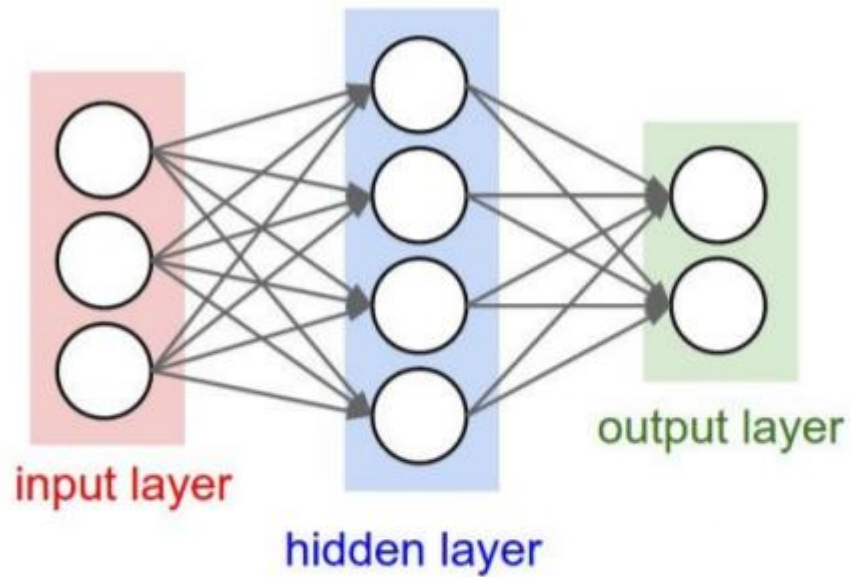
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Neural networks: Architectures

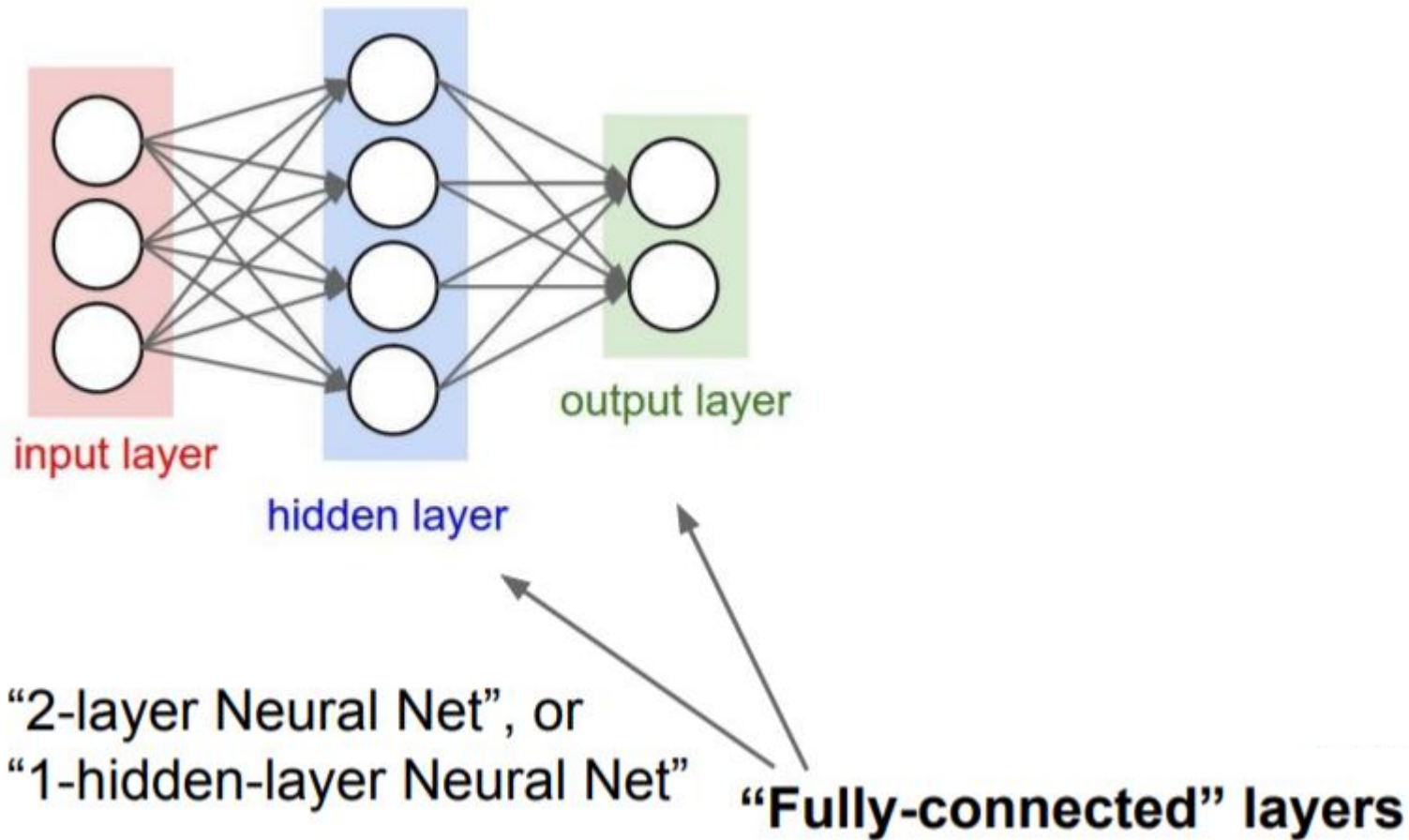


Neural networks: Architectures

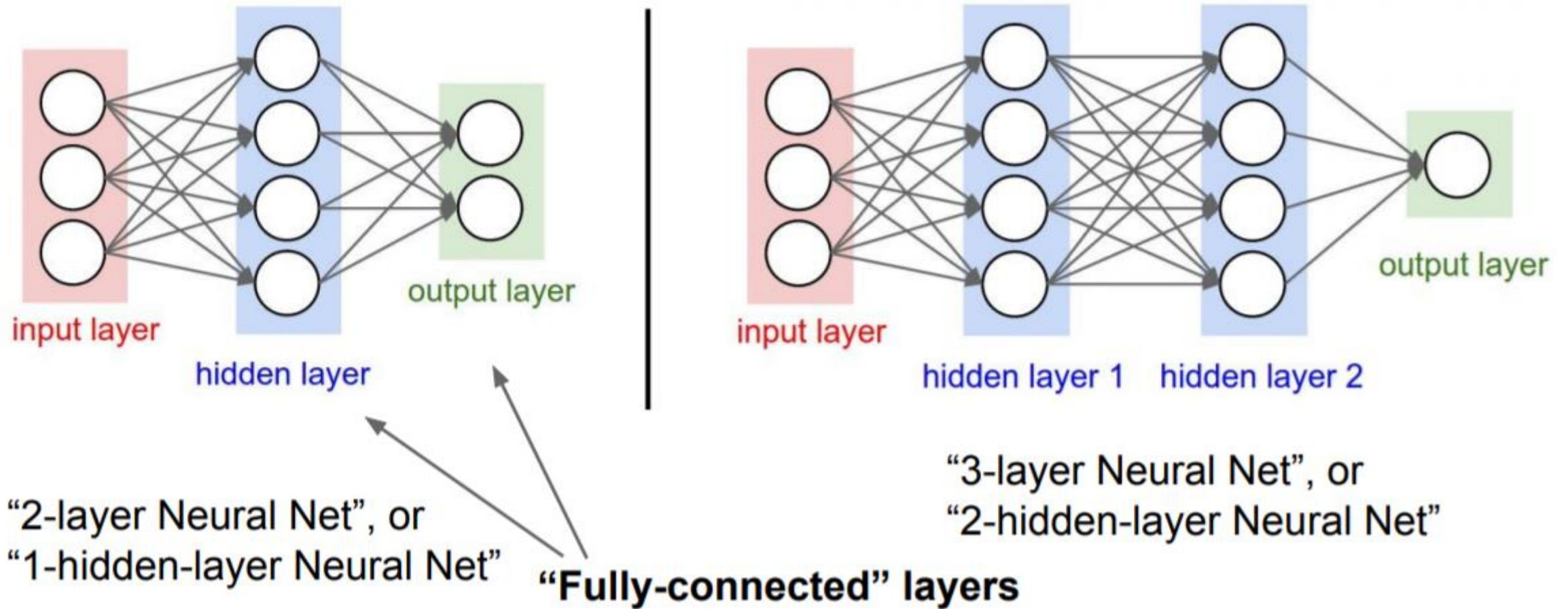


“2-layer Neural Net”, or
“1-hidden-layer Neural Net”

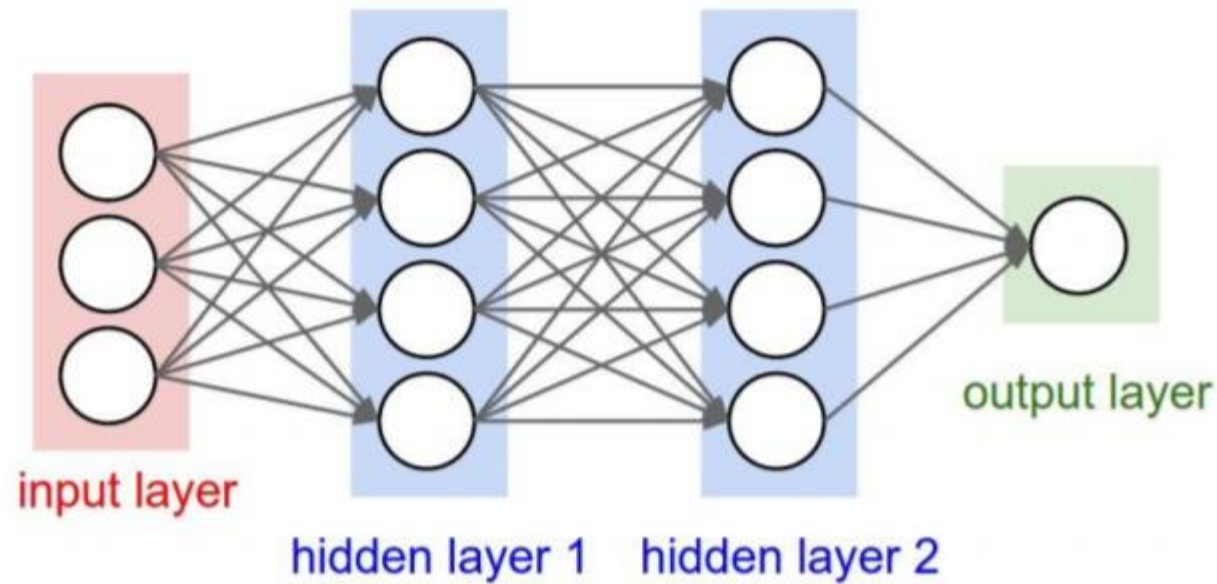
Neural networks: Architectures



Neural networks: Architectures



Example feed-forward computation of a neural network

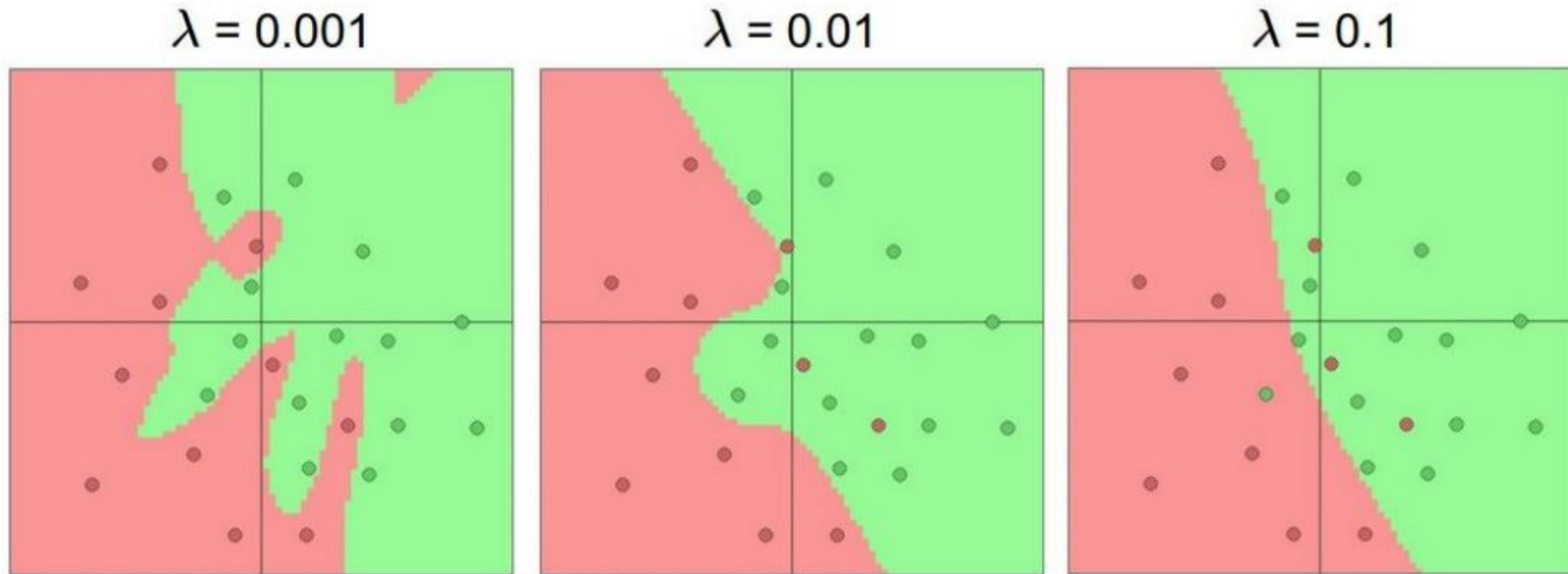


Setting the number of layers and their sizes



↑
more neurons = more capacity

Do not use size of neural network as a regularizer. Use stronger regularization instead



$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i) + \lambda R(W)$$

(Web demo with ConvNetJS:

<http://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>)

Neural Network

- Given a dataset with ground truth training pairs $[x_i; y_i]$

Neural Network

- Given a dataset with ground truth training pairs $[x_i; y_i]$
- Find optimal weights $\mathbf{W}$ using stochastic gradient descent, such that the loss function is minimized

Neural Network

- Given a dataset with ground truth training pairs $[x_i; y_i]$
- Find optimal weights $\mathbf{W}$ using stochastic gradient descent, such that the loss function is minimized
 - Compute gradients with backpropagation

Neural Network

- Given a dataset with ground truth training pairs $[x_i; y_i]$
- Find optimal weights $\mathbf{W}$ using stochastic gradient descent, such that the loss function is minimized
 - Compute gradients with backpropagation
 - Iterate many times over training set (SGD)

How to compute gradients?

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x) \quad \text{Nonlinear score function}$$

How to compute gradients?

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x) \quad \text{Nonlinear score function}$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \quad \text{SVM Loss on predictions}$$

How to compute gradients?

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x) \quad \text{Nonlinear score function}$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \quad \text{SVM Loss on predictions}$$

$$R(W) = \sum_k W_k^2 \quad \text{Regularization}$$

How to compute gradients?

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x) \quad \text{Nonlinear score function}$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \quad \text{SVM Loss on predictions}$$

$$R(W) = \sum_k W_k^2 \quad \text{Regularization}$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda R(W_1) + \lambda R(W_2) \quad \text{Total loss: data loss + regularization}$$

How to compute gradients?

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x) \quad \text{Nonlinear score function}$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \quad \text{SVM Loss on predictions}$$

$$R(W) = \sum_k W_k^2 \quad \text{Regularization}$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda R(W_1) + \lambda R(W_2) \quad \text{Total loss: data loss + regularization}$$

If we can compute $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$ then we can learn W_1 and W_2

(Bad) Idea: Derive the gradient on paper

$$s = f(x; W) = Wx$$

(Bad) Idea: Derive the gradient on paper

$$s = f(x; W) = Wx$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

(Bad) Idea: Derive the gradient on paper

$$\begin{aligned} s &= f(x; W) = Wx \\ L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\ &= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) \end{aligned}$$

(Bad) Idea: Derive the gradient on paper

$$\begin{aligned}s &= f(x; W) = Wx \\ L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\ &= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) \\ L &= \frac{1}{N} \sum_{i=1}^N L_i + \lambda \sum_k W_k^2\end{aligned}$$

(Bad) Idea: Derive the gradient on paper

$$s = f(x; W) = Wx$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1)$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda \sum_k W_k^2$$

$$= \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2$$

(Bad) Idea: Derive the gradient on paper

...

$$s = f(x; W) = Wx$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1)$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda \sum_k W_k^2$$

$$= \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2$$

$$\nabla_W L = \nabla_W \left(\frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \right)$$

(Bad) Idea: Derive the gradient on paper

Problem: Very tedious: Lots of matrix calculus, need lots of paper

$$\begin{aligned} s &= f(x; W) = Wx \\ L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\ &= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) \\ L &= \frac{1}{N} \sum_{i=1}^N L_i + \lambda \sum_k W_k^2 \\ &= \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \\ \nabla_W L &= \nabla_W \left(\frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \right) \end{aligned}$$

(Bad) Idea: Derive the gradient on paper

$$\begin{aligned}s &= f(x; W) = Wx \\ L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\ &= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) \\ L &= \frac{1}{N} \sum_{i=1}^N L_i + \lambda \sum_k W_k^2 \\ &= \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \\ \nabla_W L &= \nabla_W \left(\frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \right)\end{aligned}$$

Problem: Very tedious: Lots of matrix calculus, need lots of paper

Problem: What if we want to change loss? E.g. use softmax instead of SVM? Need to re-derive from scratch =(

(Bad) Idea: Derive the gradient on paper

...

$$s = f(x; W) = Wx$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1)$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda \sum_k W_k^2$$

$$= \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2$$

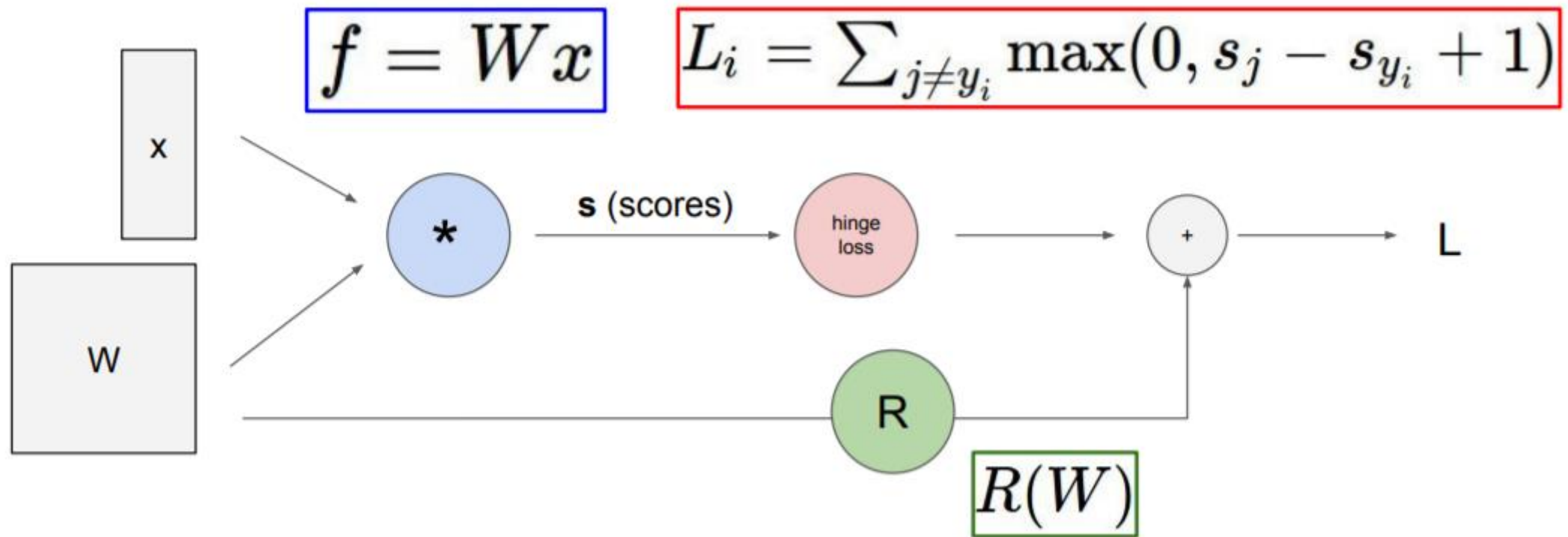
$$\nabla_W L = \nabla_W \left(\frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \right)$$

Problem: Very tedious: Lots of matrix calculus, need lots of paper

Problem: What if we want to change loss? E.g. use softmax instead of SVM? Need to re-derive from scratch =(

Problem: Not feasible for very complex models!

Better Idea: Computational graphs + Backpropagation



Computational Graphs

Computational Graphs

- Directional graph

Computational Graphs

- Directional graph
- Matrix operations are represented as compute nodes.

Computational Graphs

- Directional graph
- Matrix operations are represented as compute nodes.
- Vertex nodes are variables or operators like $+$, $-$, $*$, $/$, $\log()$, $\exp()$, ...

Computational Graphs

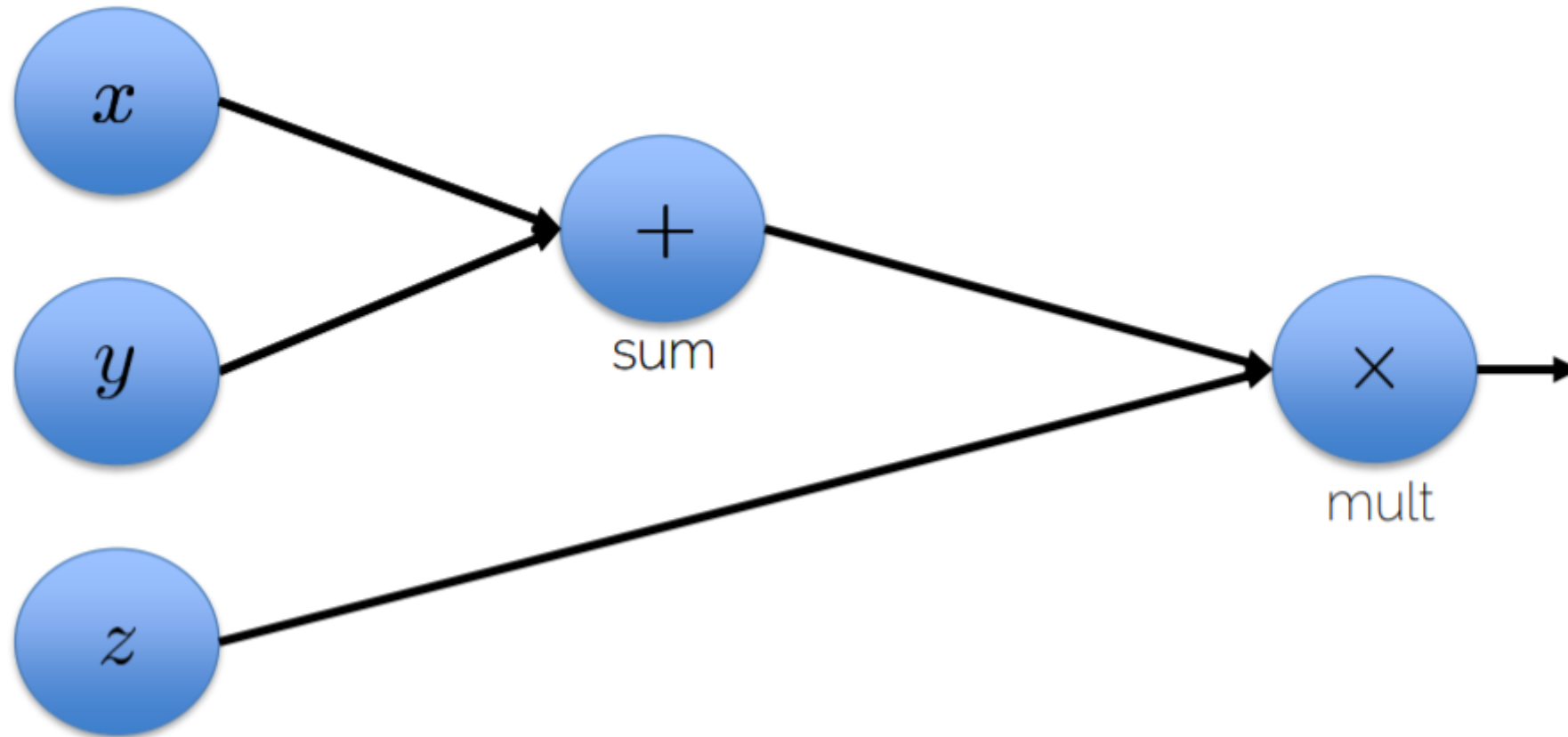
- Directional graph
- Matrix operations are represented as compute nodes.
- Vertex nodes are variables or operators like $+$, $-$, $*$, $/$, $\log()$, $\exp()$, ...
- Directional edges show flow of inputs to vertices

Computational Graphs

$$f(x, y, z) = (x + y) \cdot z$$

Computational Graphs

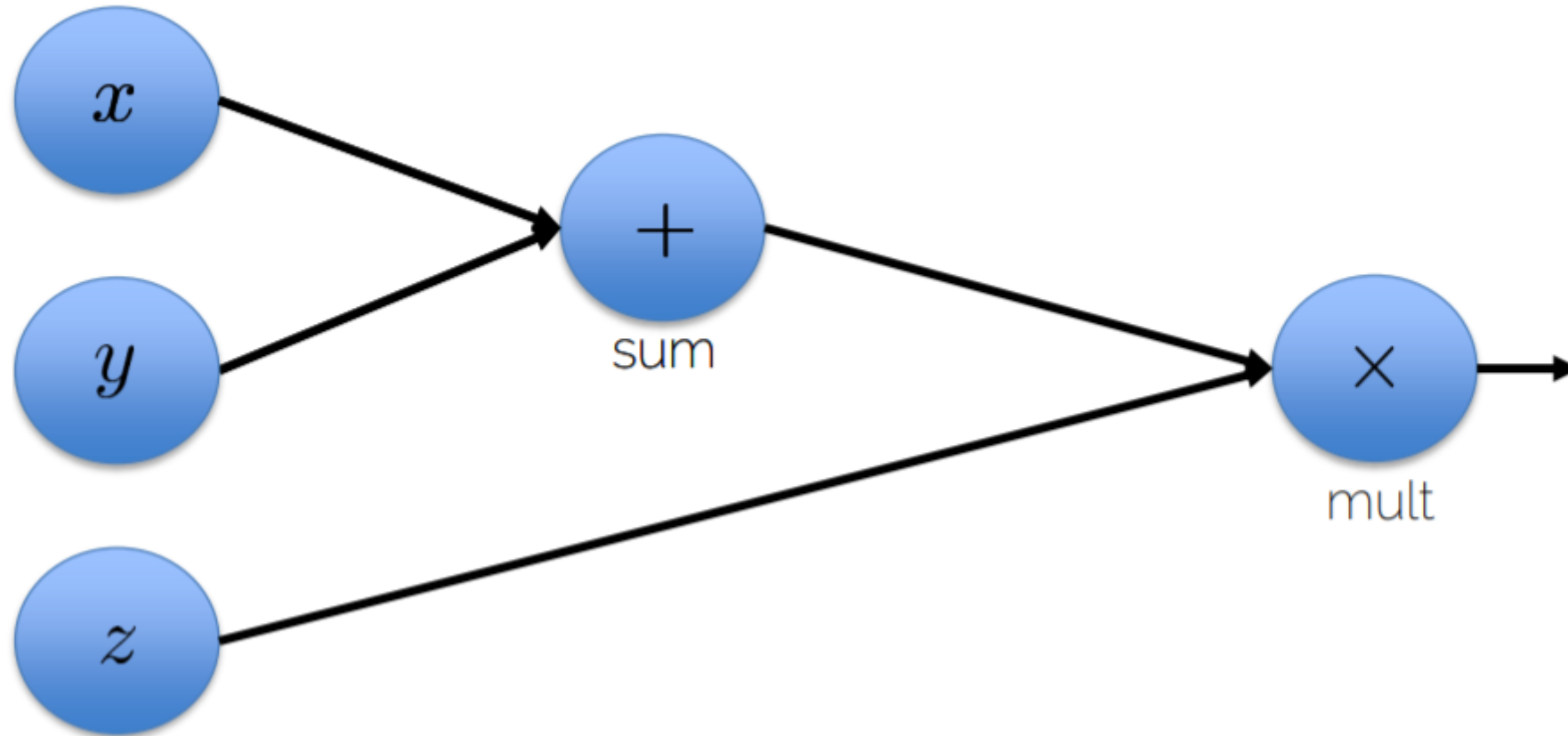
$$f(x, y, z) = (x + y) \cdot z$$



Computational Graphs

$$f(x, y, z) = (x + y) \cdot z$$

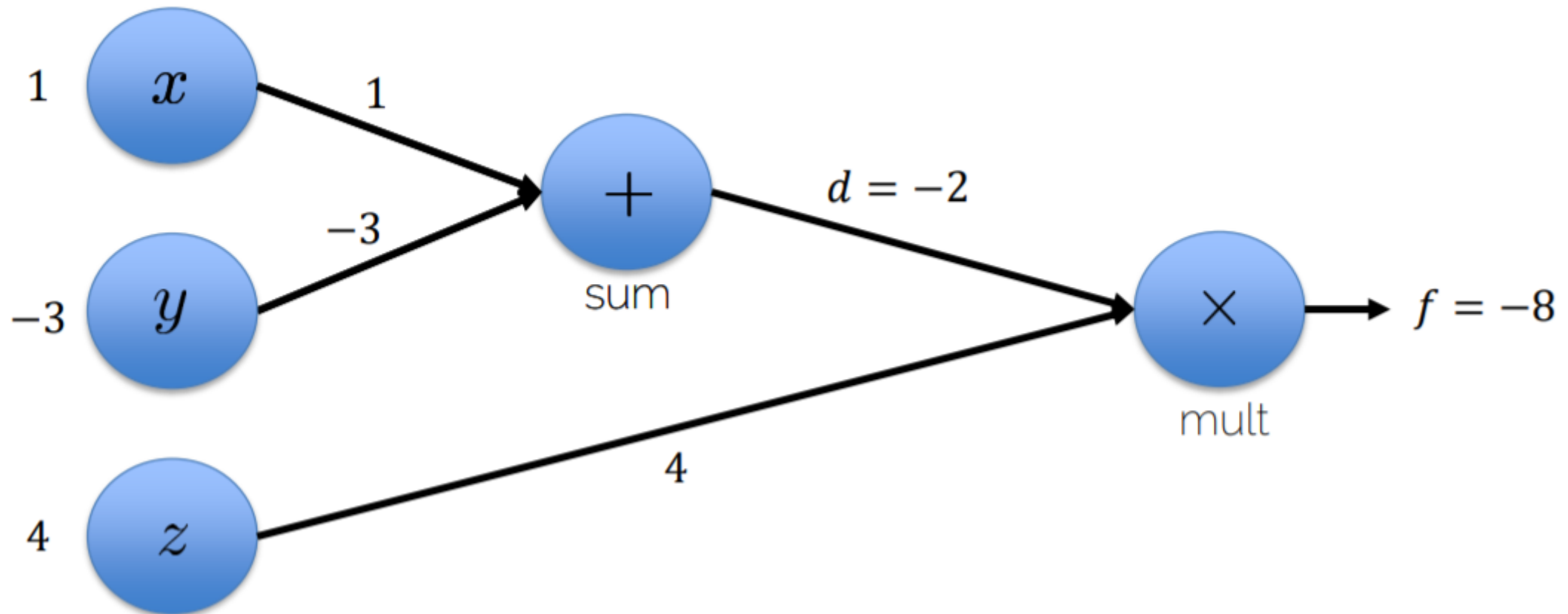
Initialization $x = 1, y = -3, z = 4$



Computational Graphs

$$f(x, y, z) = (x + y) \cdot z$$

Initialization $x = 1, y = -3, z = 4$



Computational Graphs

- Why discuss compute graphs?
- Neural networks have complicated architectures

$$\mathbf{f} = \mathbf{W}_5 \sigma(\mathbf{W}_4 \tanh(\mathbf{W}_3, \max(\mathbf{0}, \mathbf{W}_2 \max(\mathbf{0}, \mathbf{W}_1 \mathbf{x}))))$$

- Lot of matrix operations!
- Represent NN as computational graphs!

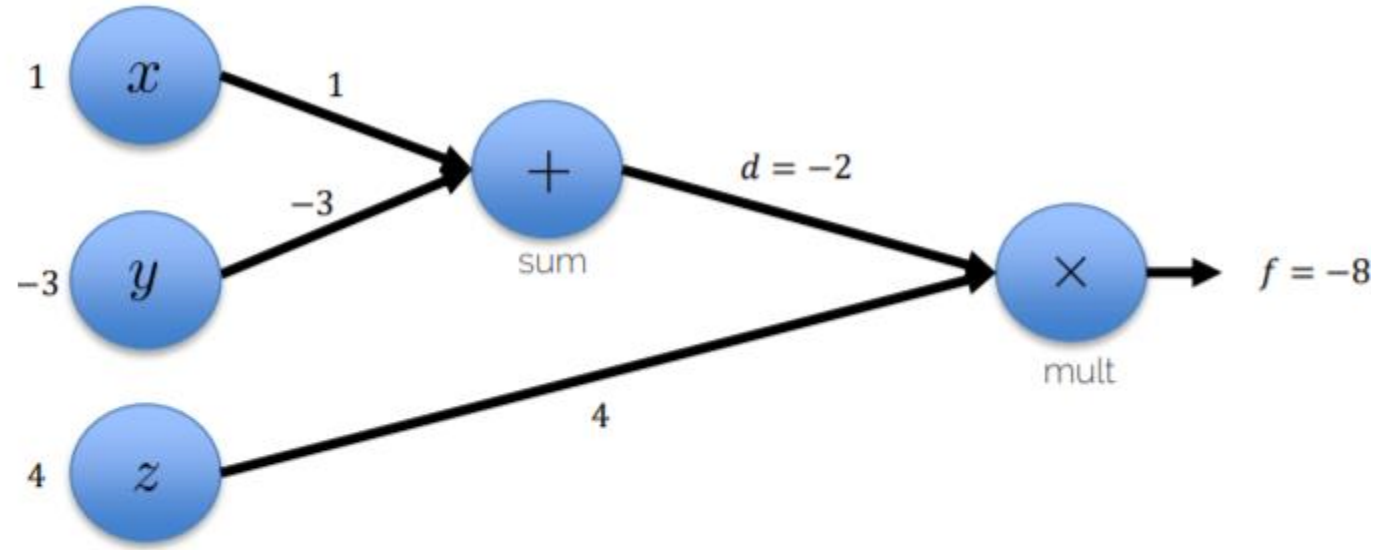
The Importance of Gradients

- Our optimization schemes are based on computing gradients
- One can compute gradients analytically but what if our function is too complex?
- Break down gradient computation

Backprop: Backward Pass

$$f(x, y, z) = (x + y) \cdot z$$

with $x = 1, y = -3, z = 4$



Backprop: Backward Pass

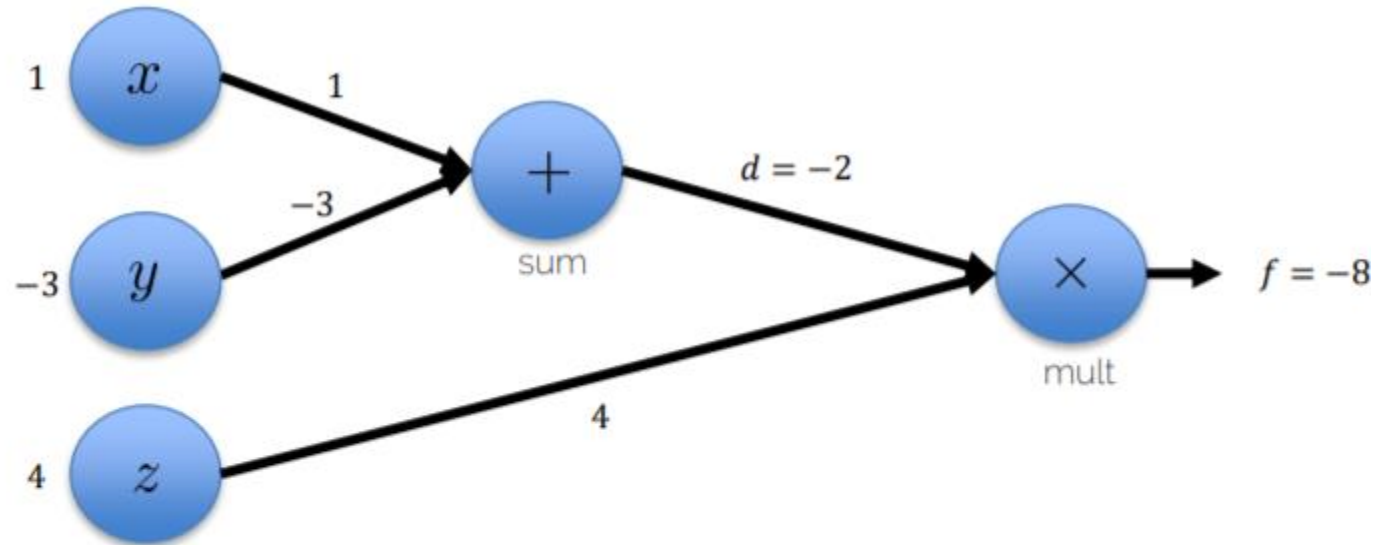
$$f(x, y, z) = (x + y) \cdot z$$

with $x = 1, y = -3, z = 4$

$$d = x + y \quad \frac{\partial d}{\partial x} = 1, \frac{\partial d}{\partial y} = 1$$

$$f = d \cdot z \quad \frac{\partial f}{\partial d} = z, \frac{\partial f}{\partial z} = d$$

What is $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$?



Backprop: Backward Pass

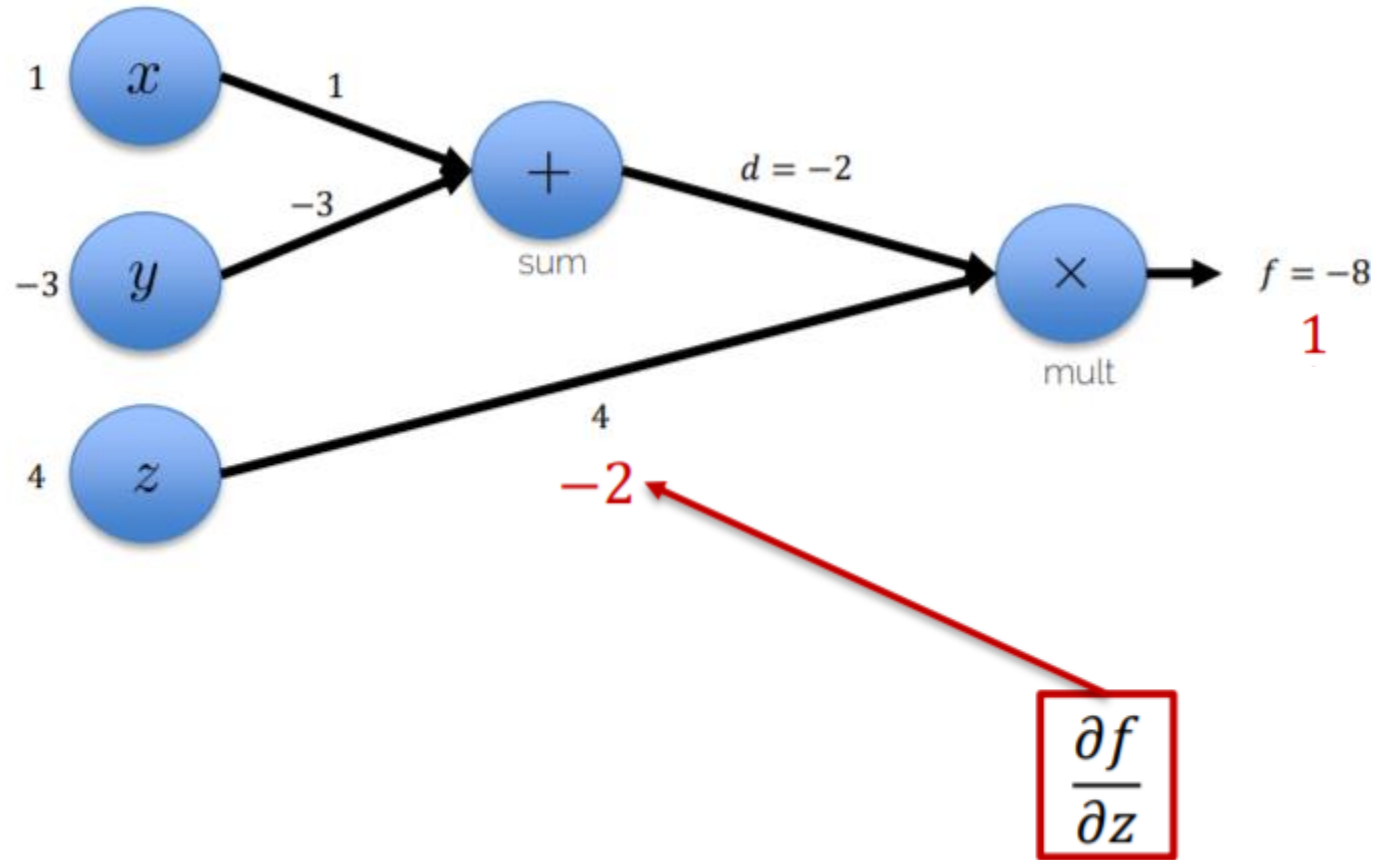
$$f(x, y, z) = (x + y) \cdot z$$

with $x = 1, y = -3, z = 4$

$$d = x + y \quad \frac{\partial d}{\partial x} = 1, \frac{\partial d}{\partial y} = 1$$

$$f = d \cdot z \quad \frac{\partial f}{\partial d} = z, \frac{\partial f}{\partial z} = d$$

What is $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$?



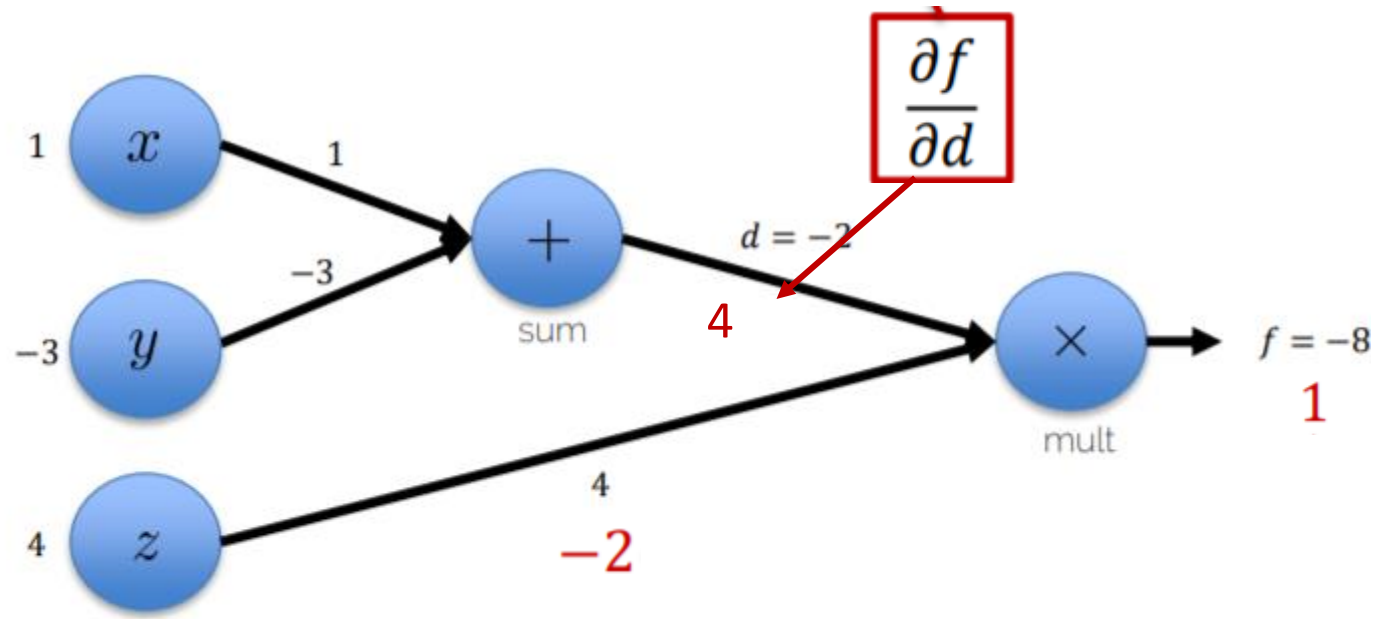
Backprop: Backward Pass

$$f(x, y, z) = (x + y) \cdot z$$

with $x = 1, y = -3, z = 4$

$$d = x + y \quad \frac{\partial d}{\partial x} = 1, \frac{\partial d}{\partial y} = 1$$

$$f = d \cdot z \quad \boxed{\frac{\partial f}{\partial d} = z} \quad \frac{\partial f}{\partial z} = d$$



What is $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$?

Backprop: Backward Pass

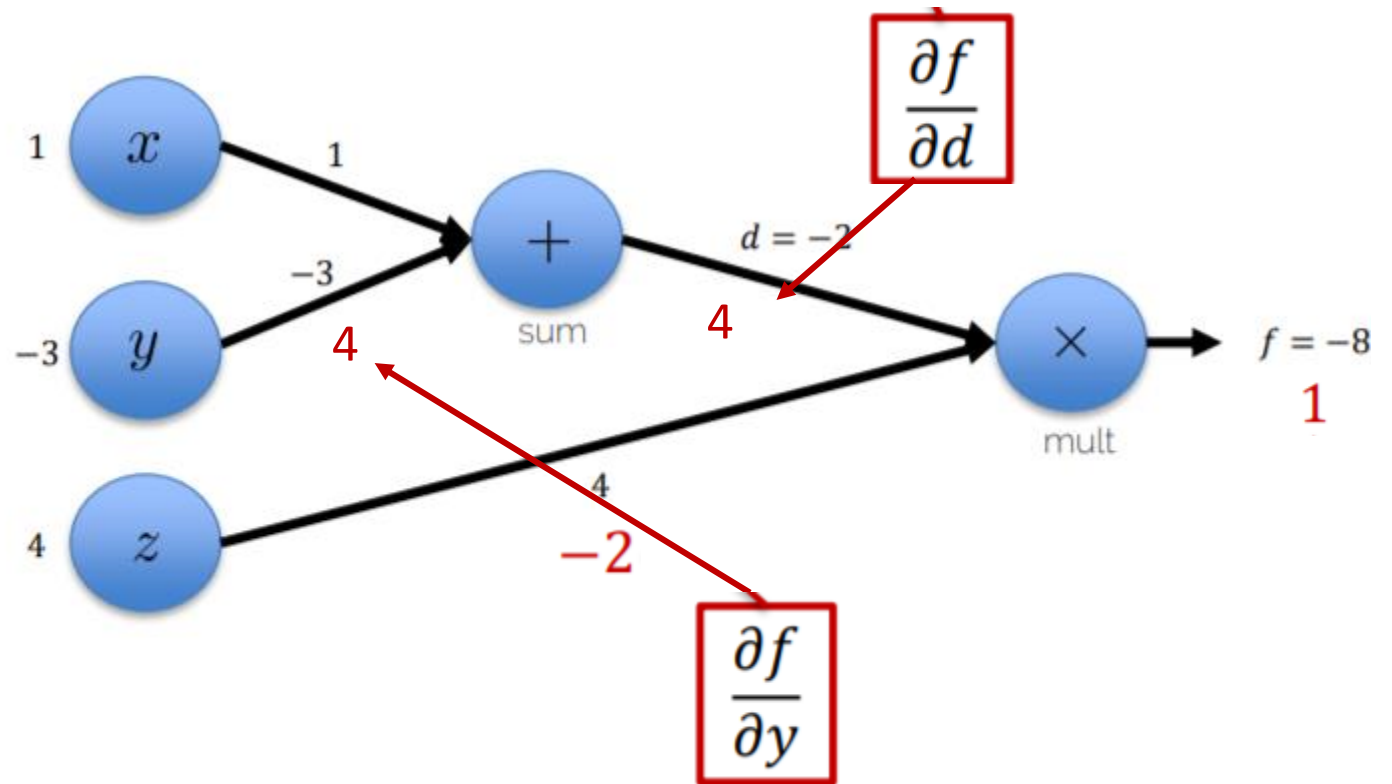
$$f(x, y, z) = (x + y) \cdot z$$

with $x = 1, y = -3, z = 4$

$$d = x + y \quad \frac{\partial d}{\partial x} = 1, \frac{\partial d}{\partial y} = 1$$

$$f = d \cdot z \quad \frac{\partial f}{\partial d} = z, \frac{\partial f}{\partial z} = d$$

What is $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$?



Backprop: Backward Pass

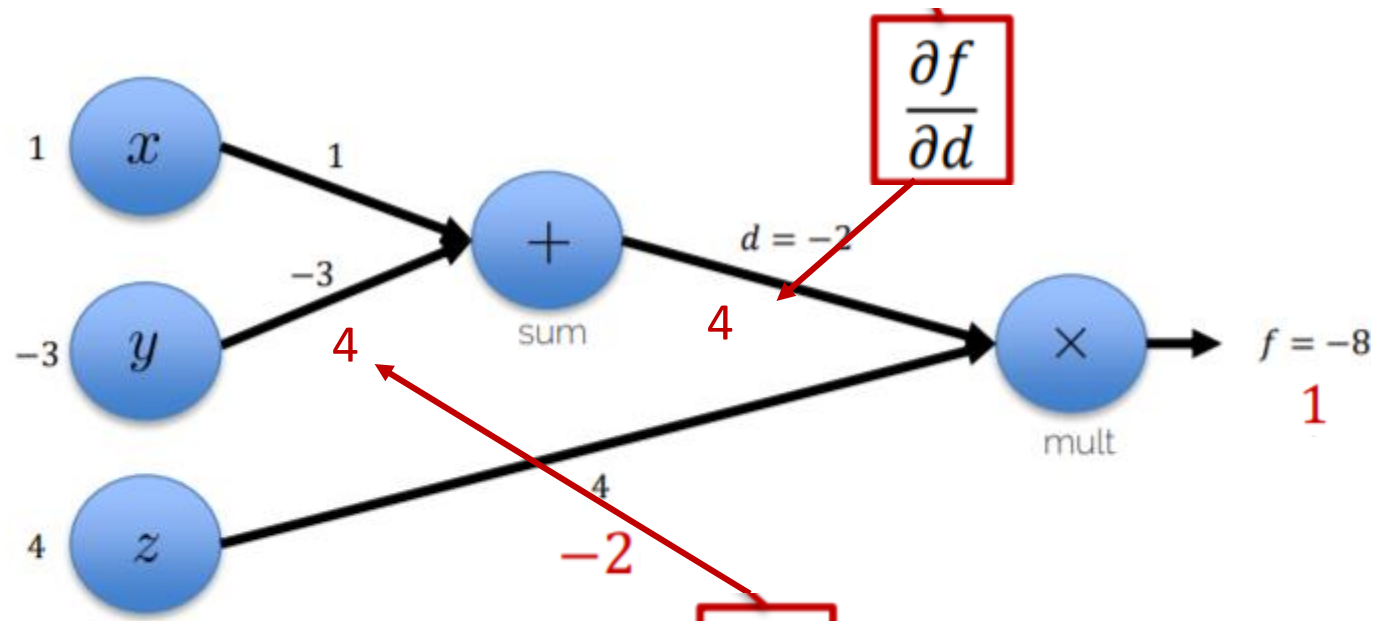
$$f(x, y, z) = (x + y) \cdot z$$

with $x = 1, y = -3, z = 4$

$$d = x + y \quad \frac{\partial d}{\partial x} = 1, \frac{\partial d}{\partial y} = 1$$

$$f = d \cdot z \quad \frac{\partial f}{\partial d} = z, \frac{\partial f}{\partial z} = d$$

What is $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$?



Chain Rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial d} \cdot \frac{\partial d}{\partial y}$$

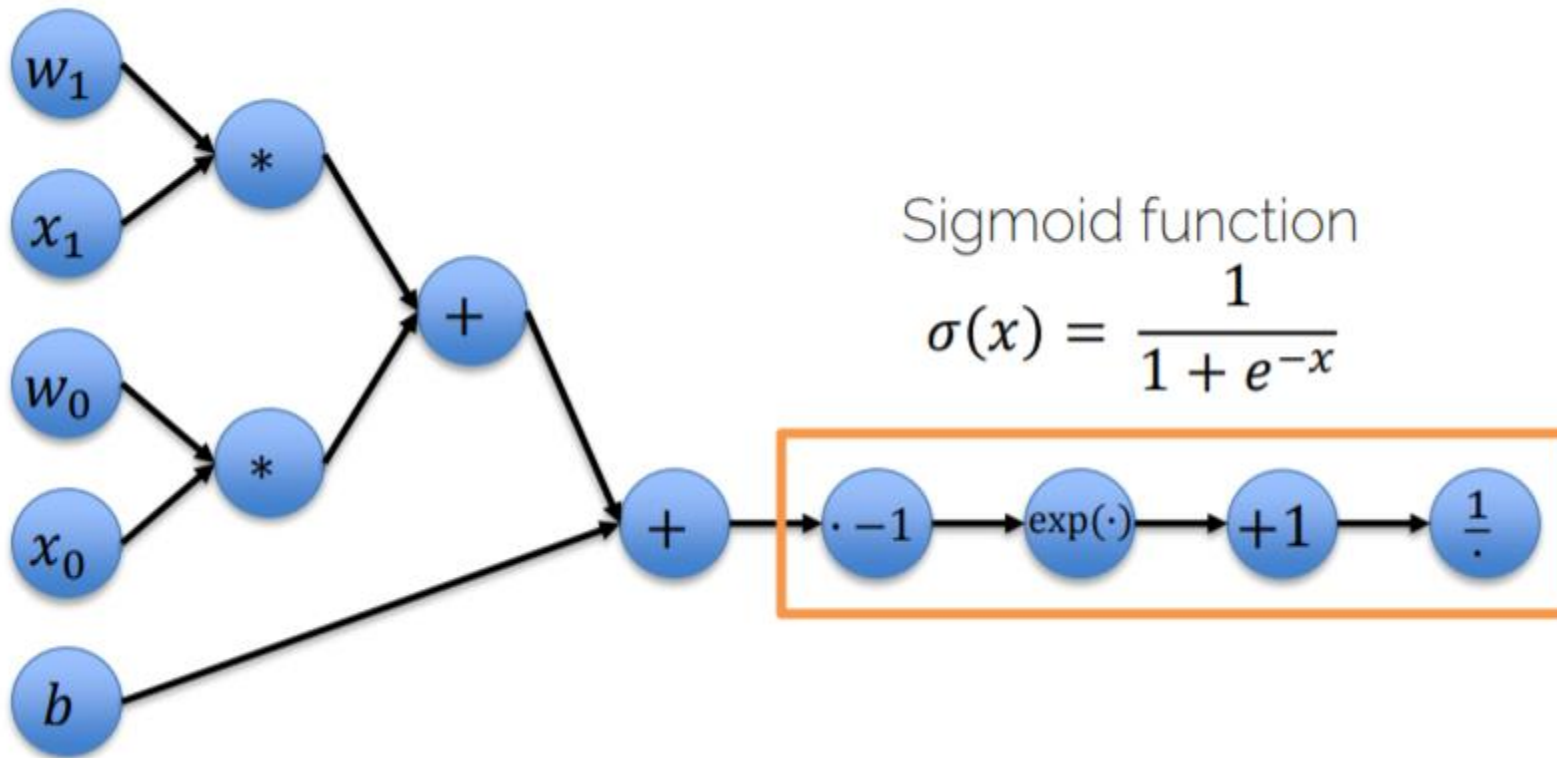
$$\rightarrow \frac{\partial f}{\partial y} = 4 \cdot 1 = 4$$

NNs as Computational Graphs

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + e^{-(b + w_0 x_0 + w_1 x_1)}}$$

NNs as Computational Graphs

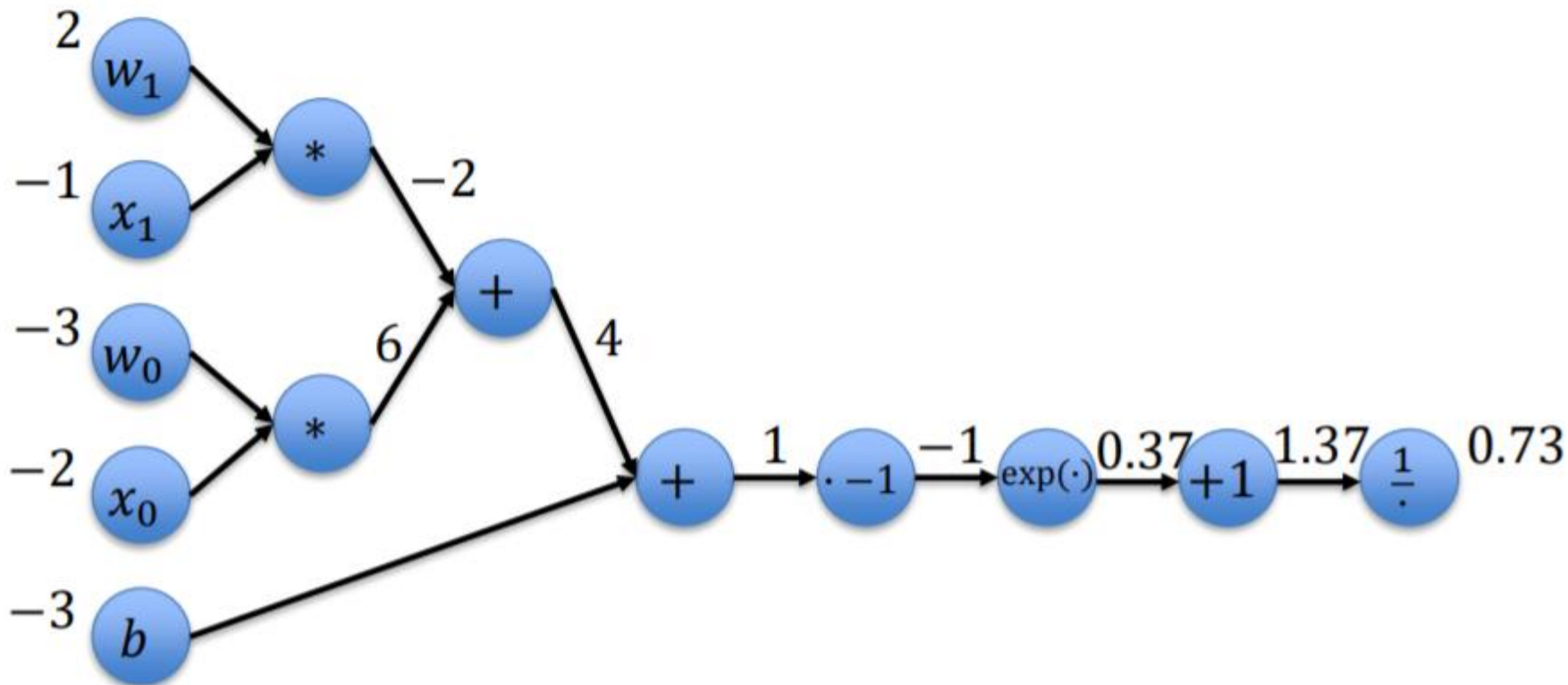
$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + e^{-(b + w_0 x_0 + w_1 x_1)}}$$



NNs as Computational Graphs (forward pass)

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + e^{-(b + w_0 x_0 + w_1 x_1)}}$$

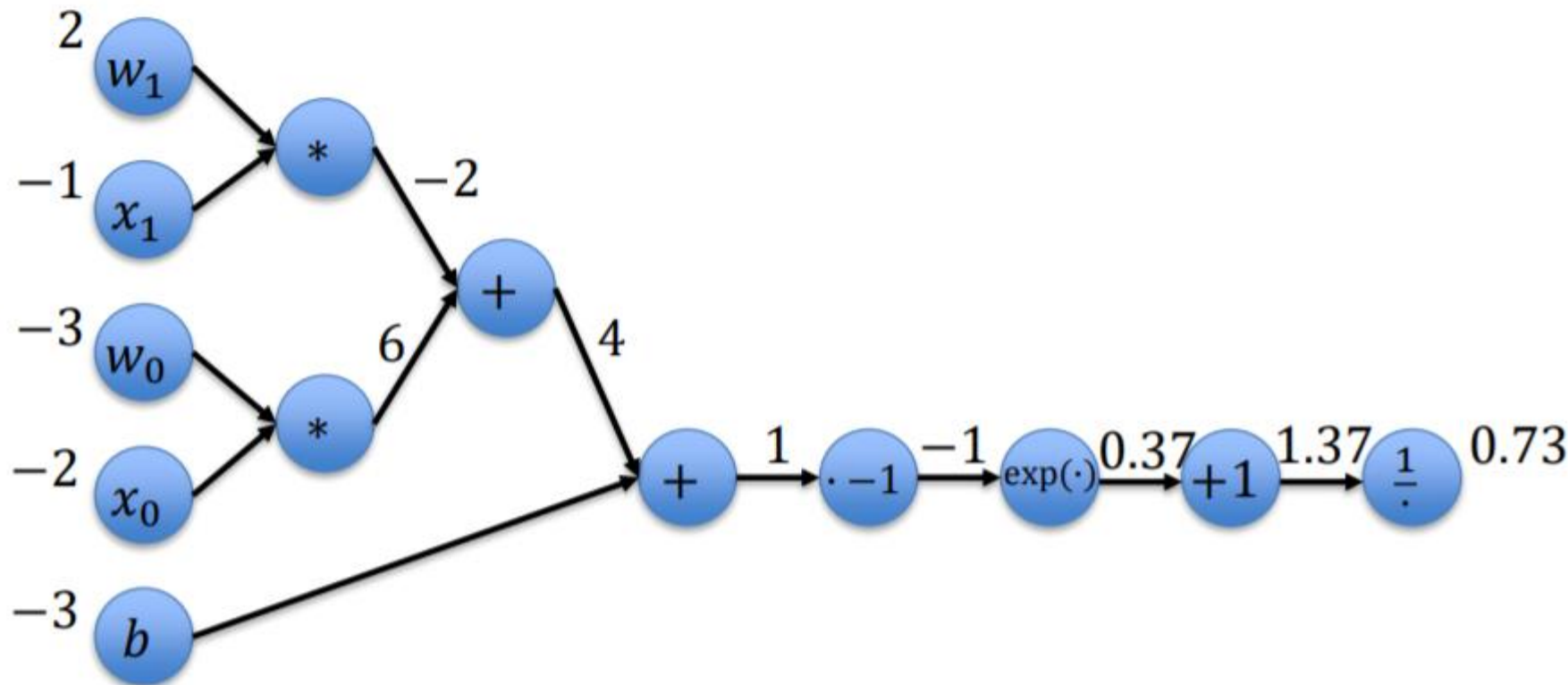
$$x_0 = -2, x_1 = -1, w_0 = -3, w_1 = 2, b = -3$$



NNs as Computational Graphs (backward pass)

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + e^{-(b + w_0 x_0 + w_1 x_1)}}$$

$$x_0 = -2, x_1 = -1, w_0 = -3, w_1 = 2, b = -3$$

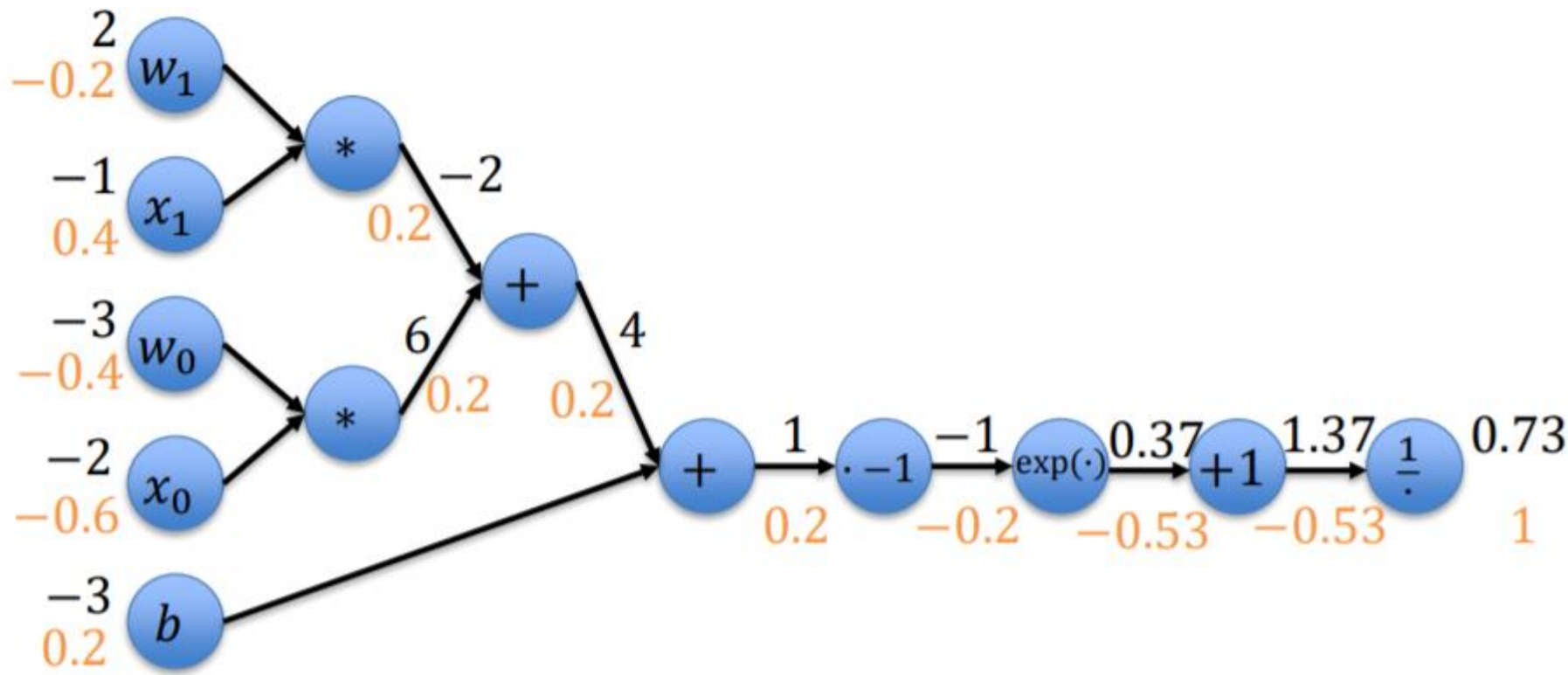


$$\begin{aligned} g(x) &= \frac{1}{x} &\Rightarrow \frac{\partial g}{\partial x} &= -\frac{1}{x^2} \\ g_\alpha(x) &= \alpha + x &\Rightarrow \frac{\partial g}{\partial x} &= 1 \\ g(x) &= e^x &\Rightarrow \frac{\partial g}{\partial x} &= e^x \\ g_\alpha(x) &= \alpha x &\Rightarrow \frac{\partial g}{\partial x} &= \alpha \end{aligned}$$

NNs as Computational Graphs (backward pass)

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + e^{-(b + w_0 x_0 + w_1 x_1)}}$$

$$x_0 = -2, x_1 = -1, w_0 = -3, w_1 = 2, b = -3$$



$$g(x) = \frac{1}{x} \Rightarrow \frac{\partial g}{\partial x} = -\frac{1}{x^2}$$

$$g_\alpha(x) = \alpha + x \Rightarrow \frac{\partial g}{\partial x} = 1$$

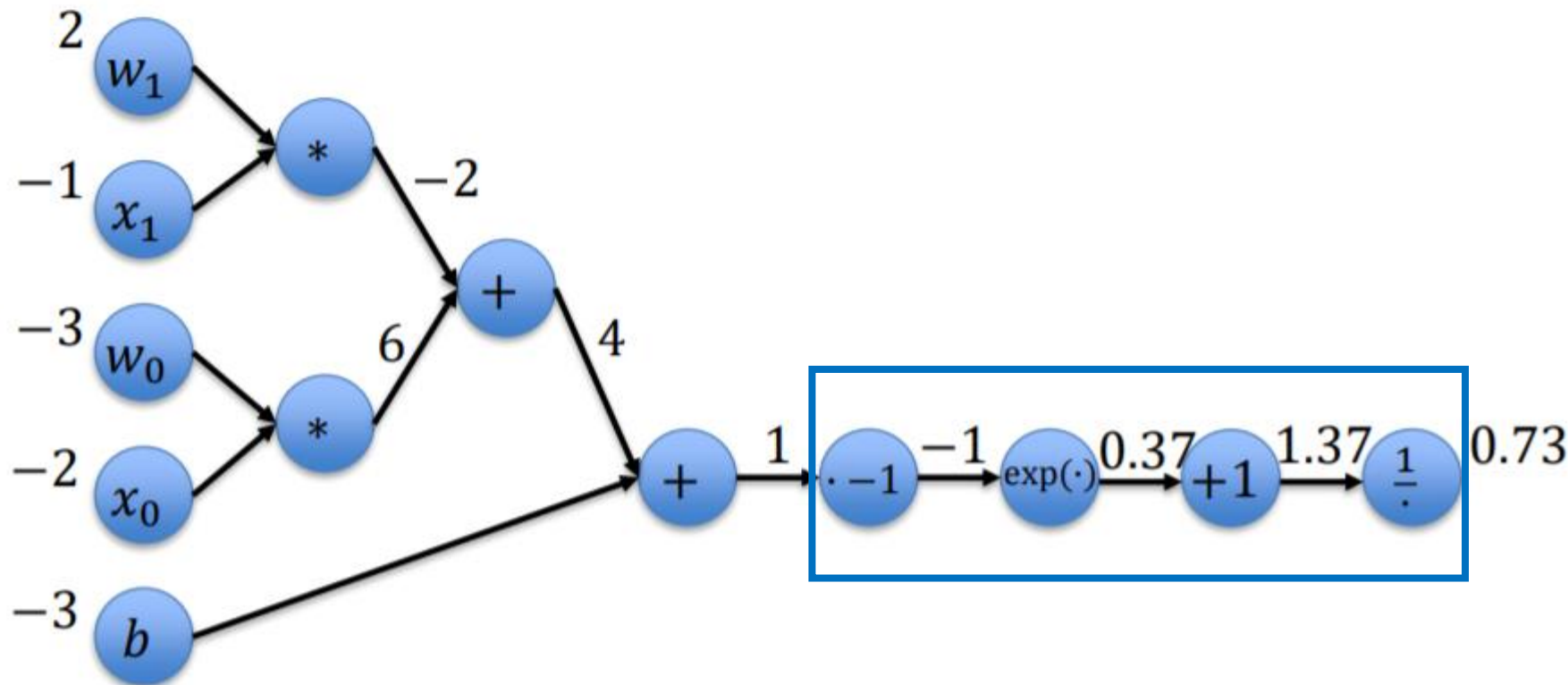
$$g(x) = e^x \Rightarrow \frac{\partial g}{\partial x} = e^x$$

$$g_\alpha(x) = \alpha x \Rightarrow \frac{\partial g}{\partial x} = \alpha$$

NNs as Computational Graphs (backward pass)

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + e^{-(b + w_0 x_0 + w_1 x_1)}}$$

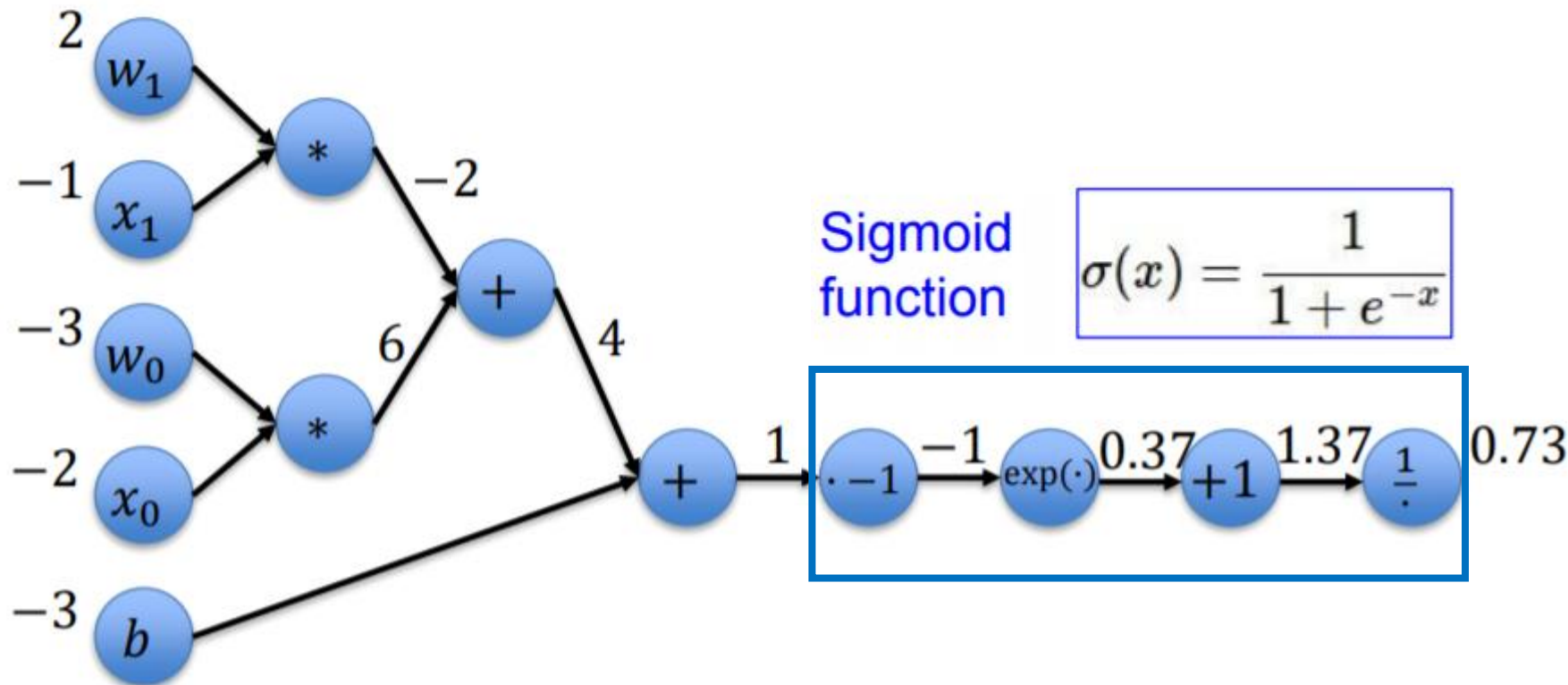
$$x_0 = -2, x_1 = -1, w_0 = -3, w_1 = 2, b = -3$$



NNs as Computational Graphs (backward pass)

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + e^{-(b + w_0 x_0 + w_1 x_1)}}$$

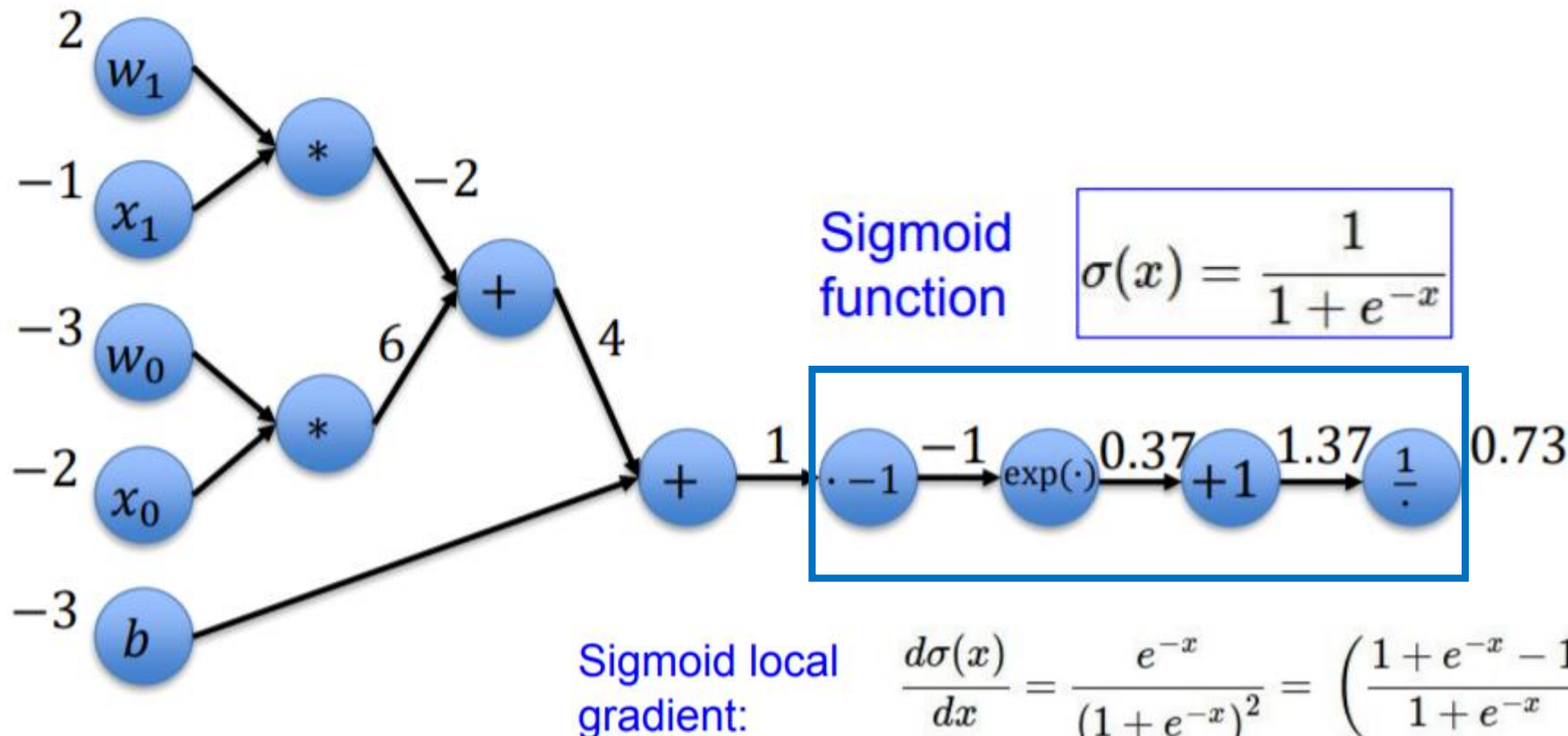
$$x_0 = -2, x_1 = -1, w_0 = -3, w_1 = 2, b = -3$$



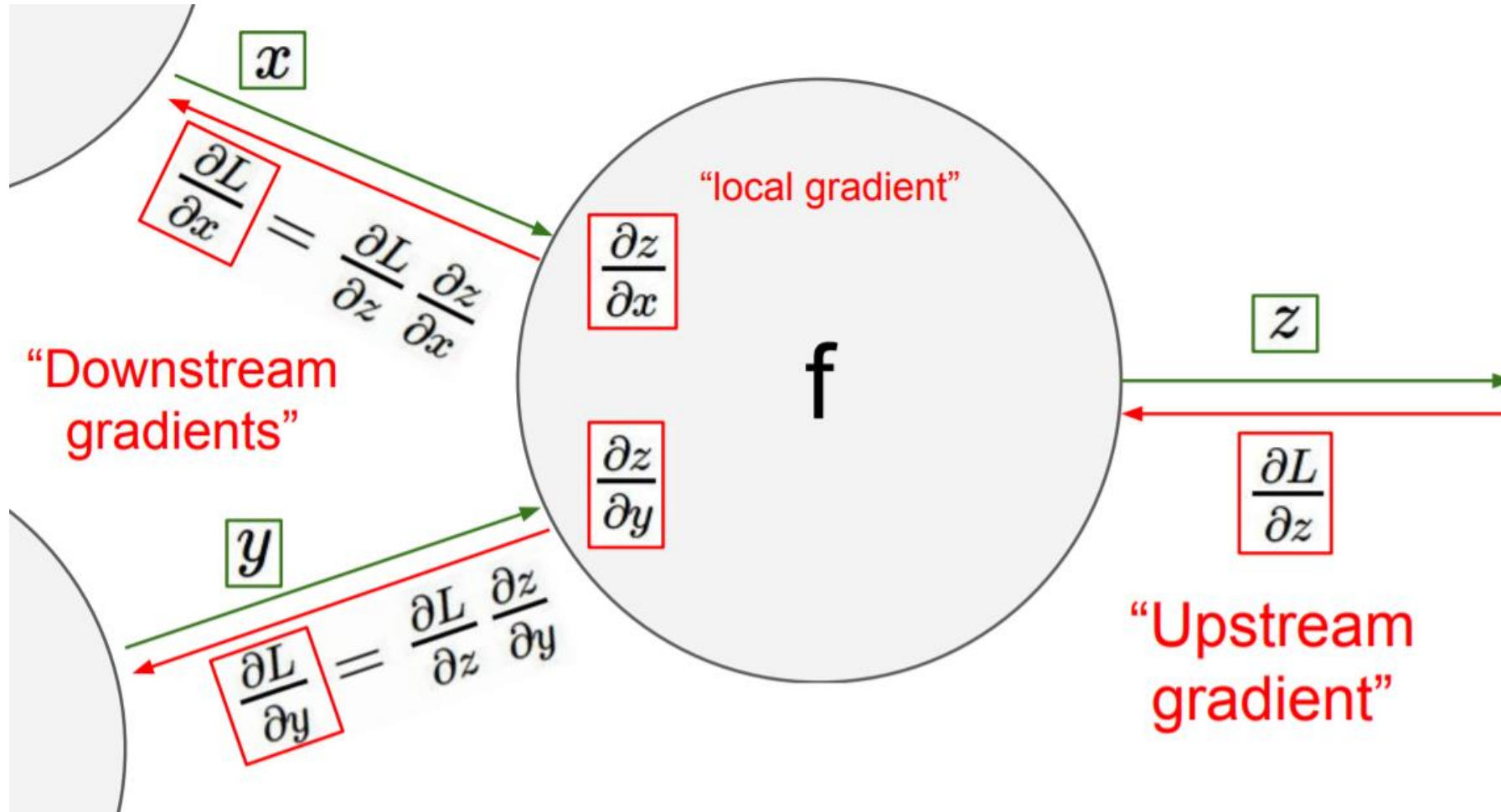
NNs as Computational Graphs (backward pass)

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + e^{-(b + w_0 x_0 + w_1 x_1)}}$$

$$x_0 = -2, x_1 = -1, w_0 = -3, w_1 = 2, b = -3$$

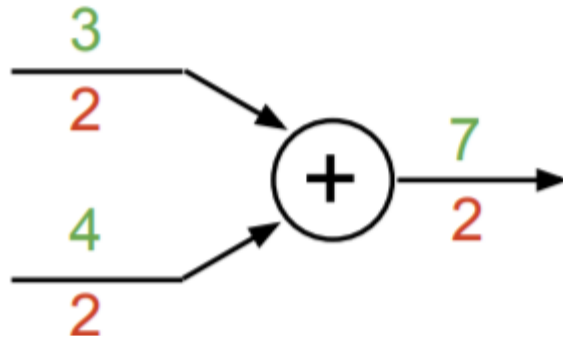


The Flow of the Gradients



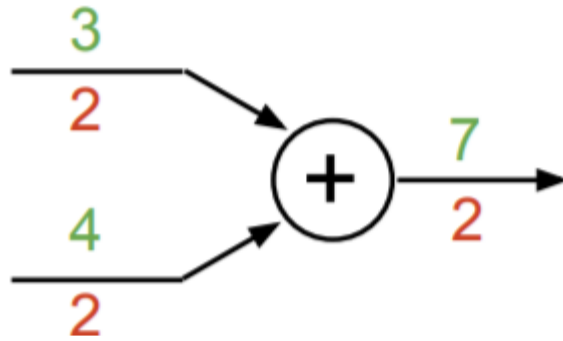
Patterns in gradient flow

add gate: gradient distributor

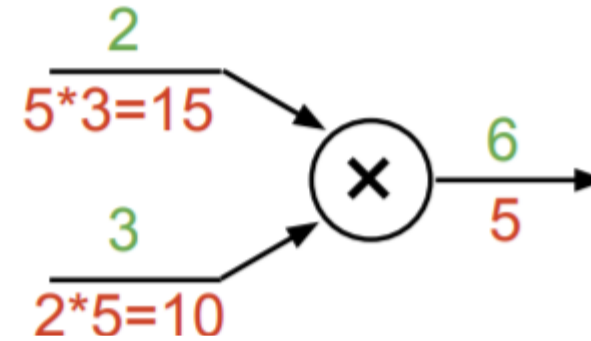


Patterns in gradient flow

add gate: gradient distributor

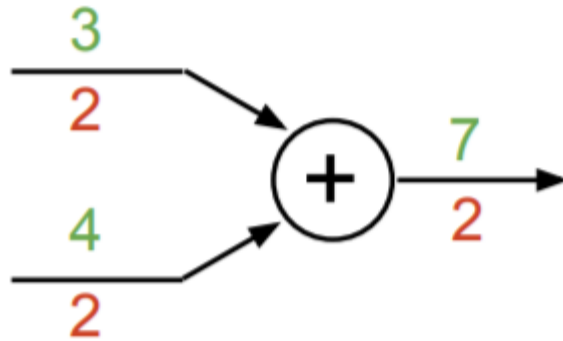


mul gate: “swap multiplier”

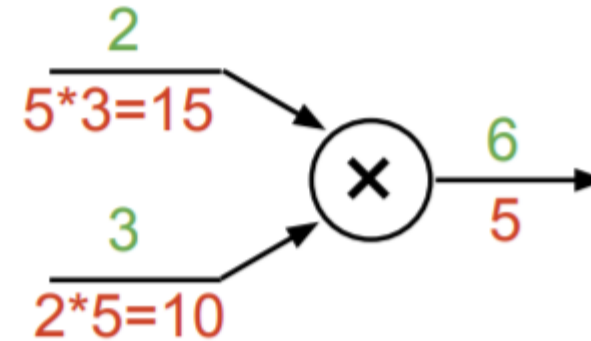


Patterns in gradient flow

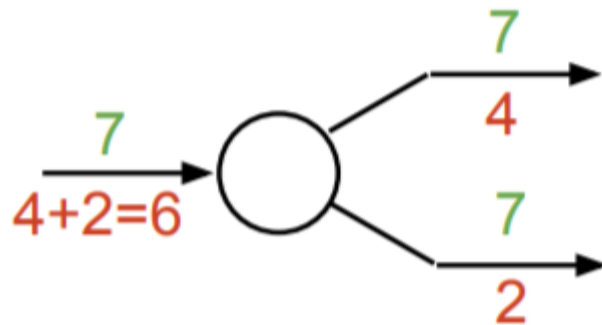
add gate: gradient distributor



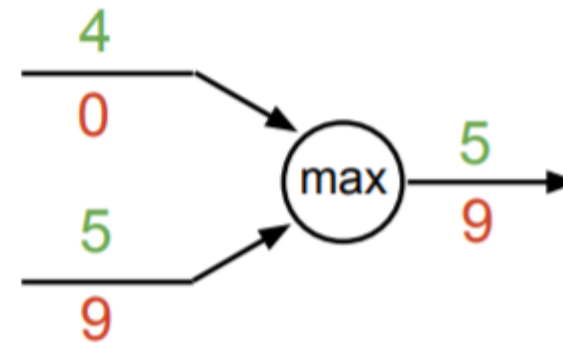
mul gate: “swap multiplier”



copy gate: gradient adder

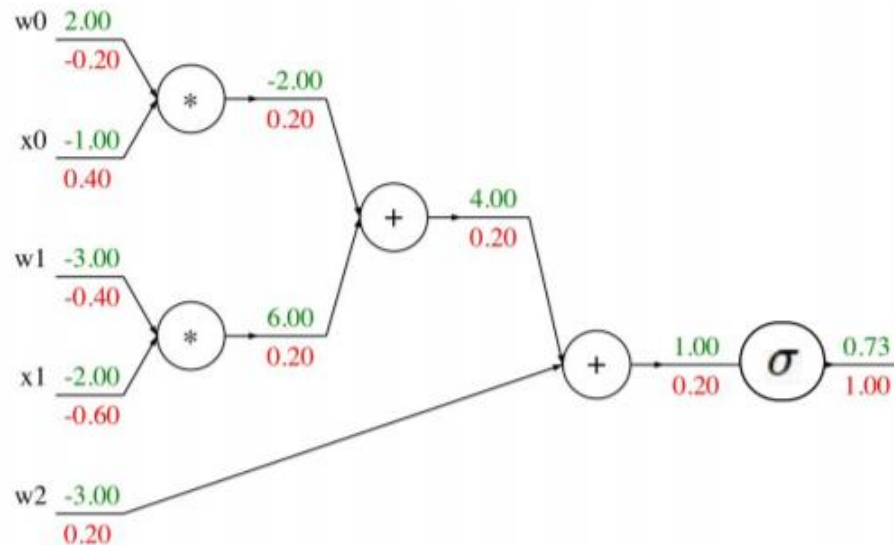


max gate: gradient router



Backprop Implementation

Backprop Implementation: “Flat” code



Forward pass:
Compute output

```
s0 = w0 * x0
s1 = w1 * x1
s2 = s0 + s1
s3 = s2 + w2
L = sigmoid(s3)
```

Backward pass:
Compute grads

```
grad_L = 1.0
grad_s3 = grad_L * (1 - L) * L
grad_w2 = grad_s3
grad_s2 = grad_s3
grad_s1 = grad_s2
grad_w1 = grad_s1 * x1
grad_x1 = grad_s1 * w1
grad_w0 = grad_s0 * x0
grad_x0 = grad_s0 * w0
```